Bridging the gap between ROS 2 and additive manufacturing: A Real-Time Control Approach for Continuous Deposition Tasks

submitted as a requirement for the degree of

Master of Engineering and Sustainable Technology Management, Industry 4.0: Automation, Robotics & 3D Manufacturing

Berlin, Germany, September 15, 2025

by

Raj Manoj Bag (3119714)

Bachelor of Mechanical Engineering, SPPU.

Primary Thesis Supervisor:	**Prof. Keskar, Ajinkya**
Associate Thesis Supervisor:	**Prof. Dr. Ratna, Kishori**

ABSTRACT

We implement a real-time 3D-printer toolpath streaming algorithm on a 6-DOF igus ReBeL robot and synchronize it with a Marlin-based polymer extruder. The coordinator aligns slicer G-code timing and semantics originally designed for buffered, planner-driven motion with a streaming robot interface (CRI), enabling phase-aligned motion and extrusion. The system is aimed to have continuous deposition enabled through the use of an E3D Revo Hemera XS direct-drive extruder, a BIGTREETECH SKR Mini E3 V3.0 controller, and Marlin. The initial process material is PETG.

We build an architecture on the ROS 2 framework with a fixed-period coordinator at approximately 50 Hz that interprets G-code (.3mf to. gcode), time-parameterizes segments, and maps the instantaneous tool speed v(t) to volumetric flow and filament feed Extrusion is streamed in relative increments (M83, G1 EΔE) using a bounded in-flight queue (8-16 lines) which advances on Marlin ok and increases the length of the queue on Marlin error. Controlling of temperature is performed with M104/M109 Safety is achieved by using M302, M112 and the EMERGENCY_PARSER, combined with a deterministic abort/recovery mechanism. CRI pose commands are sent on the robot side at the rate of one per control tick, with queue priming that helps to suppress jitter at transmission rates up to 0.2 m/s. Planar experiments represent domain primitives, such as straight line, square, spiral, lattice using 0.4 mm nozzle, typical conditions w=0.48 mm,h=0.20 mm and a speed range matched to the ReBeL.

We evaluate the synchronization policy in a bench cell where the IGUS ReBeL is rigidly mounted but capable of motion within an approximate cylindrical workspace (radius 0.2–0.6 m, height ≤ 0.35 m). Evidence consists of timing logs (acknowledgment latency, in-flight occupancy) and planar PETG coupons on a Voron build plate; motion-coupled prints are left to future work. All results use a 50 Hz coordinator and USB serial to Marlin; no fieldbus or conformal (tilted) paths are reported.

Keywords: robotic additive manufacturing, fused-filament fabrication, ROS 2, Marlin, CRI, real-time control, extrusion synchronization, igus ReBeL, Hemera XS, SKR Mini E3.

ACKNOWLEDGMENTS

I would like to express my sincerest gratitude to SRH Berlin University of Applied Sciences for providing me with the opportunity to carry out this research and for giving me first-class academic supervision throughout my studies.

I would like to extend my sincere appreciation to my supervisor, Professor Ajinkya Keskar, and associate supervisor, Prof. Dr. Ratna Kishori Kagitha, for their constant guidance, constructive criticism, and unwavering support throughout the preparation of this thesis. Their encouragement and guidance have been instrumental in shaping my research and ensuring its successful completion.

I would also like to extend my thanks to the instructors, peers, and colleagues who have added flavor to my academic journey with their input, encouragement, and discourses. Their feedback and participative knowledge base have augmented my understanding of the subject to a great extent.

Lastly, thanks to my family and friends who have stood by me, borne with my endurance, and inspired me during this challenging yet gratifying exercise.

Thanks all for your contribution to this effort.

TABLE OF CONTENTS

ABSTRACT ii
ACKNOWLEDGMENTS iii
TABLE OF CONTENTS iv
LIST OF FIGURES vi
LIST OF TABLES viii
LIST OF ABBREVIATIONS ix
List Of Symbols x
1. INTRODUCTION 1
1.1. Subject Relevance 2
1.2. Own Interest/Motivation 3
1.3. Research Gap 4
1.4. Objectives and Goals 5
1.5. Research Questions 7
1.6. Scope & Limitations 7
2. LITERATURE REVIEW & THEORETICAL FRAMEWORK 10
2.1 Introduction 10
2.2 Terminology, Scope & Boundaries 11
2.3 Historical Trajectory & Motivations 14
2.4 Robotic Polymer Extrusion Architectures 15
2.5 Toolpath Generation for Robots (Planar → Conformal) 17
2.6 Middleware & Real-Time Streaming with ROS 2/DDS 19
2.7 Firmware Semantics for Extrusion (Marlin focus) 20
2.7.1 Blocking/blocking vs. non-blocking control of temperature. 20
2.8 Process Model for Continuous Deposition 22
2.9 Measurement & Benchmarking Practices 24
2.10 Synthesis: Agreements, Disagreements, and Gaps 26
2.11 Theoretical Framework (Constructs, Assumptions, Propositions) 27
2.12 Positioning of This Thesis 30
2.13 Summary 31
3. System Architecture 32
3.1. Design Objectives and Constraints 32
3.2. Hardware Architecture 32
3.3. Electrical Integration and Wiring 34
3.4. Communication Links and Network Topology 36
3.5. Software Stack (High-Level) 37
3.6. Coordinate Frames, TCP, and Workpiece Setup 39
3.7. Safety Envelope and Interlocks 40
3.8. Default Configuration for Experiments 41
3.9. Risk Register (System-Level) 42
4. METHODOLOGY 44
4.1. Methodological Overview and Assumptions 44
4.2. Calibration and Validation 44
4.3. Toolpath Timing and Segment Parameterization 45
4.4. Synchronization Policy (Host-Streamed, Acknowledgment-Aware) 46
4.5. Driver and Firmware Interaction 47
4.6. Handling Transients at Corners and Stops 48
4.7. Safety and Abort Behavior 49
4.8. Data Logging and Metric Extraction 49
4.9. Experimental Protocols 51

4.10. Validity and External Checks ... 51
4.11. Visual Pipeline and Timing Diagram ... 51
5. FINDINGS ... 53
5.1. Scope of captured evidence ... 53
5.2. Methods (Summary of runs) ... 54
5.3. Timing Results ... 54
5.4. Summary of Observations ... 58
5.5. Figure ... 60
5.6. Limitation & Implication ... 60
5.7. Cost of Printing ... 61
6. DISCUSSION of findings ... 64
6.1. Short Summary ... 64
6.2. What the results mean w.r.t. objectives ... 64
6.3. Comparison with related work ... 65
6.4. Methodological limitations and validity ... 65
6.5. Transferability and practical implications ... 66
6.6. Where our findings align or diverge from the literature ... 66
6.7. Actionable recommendations ... 66
6.8. Future Work ... 67
7. CONCLUSION ... 68
APPENDIX A - ROS 2 package ... - 1 -
APPENDIX B – G-code & ANALYSIS ... - 5 -
ALPHABETICAL LIST OF REFERENCES ... - 8 -
Legal Caution ... III

LIST OF FIGURES

Figure 1 System overview: igus ReBeL with E3D Revo Hemera XS and SKR Mini E3 V3.0 controller; host runs ROS 2 coordinator. This figure anchors the scope of the platform used throughout the thesis..... 3

Figure 2 Path Overview..... 7

Figure 3. Thesis Scope vs. Out of Scope..... 8

Figure 4 Embedded Robot (yaun yao, 2021)..... 11

Figure 5 Printing Process with Robotic Arm (yaun yao, 2021) 12

Figure 6. Volumetric Flow Rate Q(t) (created with assistance from AI software) 23

Figure 7. E3D Revo Hemera XS at the tool tip..... 33

Figure 8. BIGTREETECH SKR Mini E3 V3.0 34

Figure 9. Fixturing & Safety: (a)LAN Wire, (b) external emergency-stop, (c) Clamps, (d) Base Plate, (e) M6 Bolts for Base Plate, (f) Fixed igus Robotic 6Dof arm..... 34

Figure 10. Schematic Diagram of SKR Board 35

Figure 11 ROS 2 Topology (rqt_graph) 38

Figure 12 Topic echo (Heartbeat & In-flight) 39

Figure 13. Safety layers and transition map (PAUSE/ABORT paths)..... 41

Figure 14. Coordinator tick loop 47

Figure 15 Batch runner producing CSVs 50

Figure 16. End-to-end pipeline..... 52

Figure 17 Analysis log..... 53

Figure 18 window 30 in-flight..... 55

Figure 19 window 60 in-flight..... 55

Figure 20 window 30 latency 55

Figure 21 window 60 latency 55

Figure 22 Naive 30 In-flight 2

Figure 23 Naive 60 In-flight 2

Figure 24 Naive 30 Latency 2

Figure 25 Naive 60 Latency 2

Figure 26 ROS 2 In-flight 58

Figure 27 Ack latency histogram (ROS 2 windowed run). 59

Figure 28 Qualitative concentric square 60

Figure 29 square with diagonal infill 60

LIST OF TABLES

Table 1. Comparison of Cartesian/Gantry FFF and Robot-Mounted Extrusion (6-DOF) 4

Table 2. Gap Matrix..... 13

Table 3. Comparative Systems (Robot, Extruder, Control Split, Link, Sync Method, Notes) 15

Table 4. Firmware Command Semantics 22

Table 5. Bill of materials and key spec 34

Table 6. SKR Connection Map 36

Table 7. Power budget and fusing 36

Table 8. Communication settings 37

Table 9 TCP and frame parameters 40

Table 10 Safety thresholds and runtime guards used in experiments. 41

Table 11. Default parameter set..... 42

Table 12 Risk register and mitigations 43

Table 13. Calibration outcomes..... 45

Table 14. Logging schema (CSV) 50

Table 15 Timing & queueing metrics (extruder-only bench)..... 57

LIST OF ABBREVIATIONS

Abbrev.	Meaning / Expansion	Note
AM	Additive Manufacturing	—
FFF	Fused Filament Fabrication	Polymer extrusion AM
WAAM	Wire + Arc Additive Manufacturing	Architectural analogue only
DOF	Degrees of Freedom	6-DOF robot
TCP	Tool Center Point	At nozzle tip
ROS 2	Robot Operating System 2	Host framework
DDS	Data Distribution Service	ROS 2 middleware standard
QoS	Quality of Service	ROS 2 reliability/history/deadlines
RMW	ROS 2 middleware implementation	e.g., rmw_fastrtps, rmw_cyclonedds
CRI	Commonplace Robotics Interface	Robot TCP driver/protocol
ALIVEJOG	CRI keep-alive message	Prevents session timeout
SKR	BIGTREETECH SKR Mini E3 V3.0	Extruder controller board
USB CDC	USB Communications Device Class	Marlin serial link (115200 baud)
PID	Proportional–Integral–Derivative	Hot end temperature control
E-steps	Extruder steps per mm	Calibrated (Marlin M92)
PLA	Polylactic Acid	Material
PETG	Polyethylene Terephthalate-Glycol	Material
G-code	RS-274/NGC motion language	G0/G1, etc.
M-code	Misc. control commands	M104, M109, M302, M112, etc.
ok	Marlin acknowledgment token	Parsed by host
CSV	Comma-Separated Values	Log format
p95 / p99	95th / 99th percentile	Latency/metrics reporting
VM	Virtual Machine	Host environment notes
PSU	Power Supply Unit	24 V rail
TMC2209	Trinamic stepper driver	On SKR E0
HE0 / T0	Hot end heater / Thermistor ports	On SKR
FAN0 / FAN1	Part-cooling / Heatsink fan headers	On SKR

LIST OF SYMBOLS

Symbol	Meaning	Units
w	Line width	mm
h	Layer height	mm
$v(t)$	Tool speed at TCP	mm/s
s	Path length parameter	mm
d_f	Filament diameter	mm (1.75)
$A_f = \pi(d_f/2)^2$	Filament cross-section area	mm^2 (≈2.405 for 1.75 mm)
$Q(t) = w \cdot h \cdot v(t)$	Volumetric flow	mm^3/s
$\dot{E}(t) = Q/A_f$	Filament feed rate	mm/s
F	G-code feed (motion/extrusion)	mm/min
Δt	Control period / coordinator tick	s (e.g., 0.020)
$\Delta E_k = \dot{E} \cdot \Delta t$	Extrusion increment per tick	mm
$N(t)$	In-flight unacknowledged commands	lines
N_{min}, N_{max}	Bounds of in-flight window	lines (e.g., [8,12])
L_{ok}	Acknowledgment latency (t_ok − t_send)	ms
T_{nozzle}	Nozzle temperature setpoint	°C
S_{fan}	Part-cooling fan duty	% (0–100)
A_{max} , Vmax	Kinematic limits (accel/speed)	mm/s^2, mm/s
Θ_{corner}	Interior path angle	deg
κ	Curvature of path	1/mm
ρ_m	Material density (if used for mass)	g/cm^3

1. INTRODUCTION

The geometric and workspace constraints of Cartesian gantries are challenged in multi-axis robotic additive manufacturing (AM), where fused-filament fabrication (FFF) is used. A 6-DOF arm can orient the nozzle in continuous spatial paths, apply the extrusion process to realize inclined surfaces or curved surfaces, and lessen support elements in printing, which has been proven in a number of studies in autonomous robotic FFF and around-the-body/curved-layer research (Yao, 2021; Lu, 2025). Fulfilling that promise is not trivial: Commercial slicers output buffered, planner-based G-code to extrusion controllers, whereas industrial arms process streamed motion instructions with their own timing and queueing. Lack of strict coordination causes the phase between the tool accelerations/corners drifting, resulting in under-/over-extrusion and defects visible in the products (Coogan & Kazmer, 2019).

The thesis aims to achieve synchronization of an igus ReBeL 6-DOF stage (TCP velocity < 0.2 m/s) via a coordinator implemented using ROS 2 that will translate slicer G-code into two streams coupled in the system: fixed period poses of the robot (CRI), and relative extrusion distances to a Marlin-controlled E3D Revo Hemera XS (1.75 mm) on a BIGTREETECH SKR Mini E3 V3.0. The volumetric flow and filament feed are plotted as a derivative of instantaneous tool speed $v(t)$.

$$Q(t) = whv(t), \quad E_{rate}(t) = \frac{Q(t)}{A_f}, \qquad A_f = \pi\left(\frac{d_f}{2}\right)^2$$

with integration $\Delta Ek = Erate(tk)\Delta t$ to produce the relative extrusion commands.

The serial connection uses a bounded in-flight queue sealed with Marlin ok acknowledgments to mitigate the middleware/USB jitter experienced in ROS 2/DDS chains (Kronauer, 2021; Coutinho, 2023). We test line, square, spiral, and lattice primitives on planar surfaces in PETG with 0.4 mm nozzle, including measurements of statistic bead-width versus commanded Q, path-tracking error against speed, and occupancy/acknowledgment latency.

1.1. Subject Relevance

Robotics-enabled additive manufacturing has emerged as a revolutionizing field in the advanced manufacturing research, providing a greater freedom in design, scalability, and flexibility compared to traditional 3D printing machines. Unlike Cartesian 3D printers where robotic arms are capable of creating complex toolpaths in multi-axis, non-planar deposition which is very essential in the manufacturing functionally graded their materials, printing irregular surfaces and reducing support material (Ding, 2016; Pan, 2019).

In recent years, ROS 2 is increasingly used for industrial robotic AM because its executor/QoS model exposes timing controls needed for streaming (Macenski, 2022). ROS 2 introduces the middleware which is needed for synchronized motion control, sensor integration and real-time decision making. But to make it possible to achieve complete functional capability for extrusion-based deposition, the robot system must be combined with a specially designed extrusion control subsystem capable of managing filament feed rate, hot end temperature, and cooling normally managed by single-purpose 3D printer controller boards such as the BIGTREETECH SKR Mini E3 V3.0.

The chosen extruder, E3D Revo Hemera XS, is lightweight and small in size, featuring precise filament handling suitable for PLA, PETG, and other such thermoplastics. Its usability with the SKR controller improves the extrusion control and allowing the extruder to receive synchronized G-code commands while the IGUS ReBeL 6DOF robot can execute complex motion paths in 3D space very smoothly. This integration of two systems ROS 2 for planning motion and extrusion driven by SKR closes a significant robotic additive manufacturing gap, enabling the deposition of continuous material paths on any surface with high spatial accuracy (García de Soto, 2018).

On the industrial front, this would coincide with the shift toward Industry 4.0, flexible manufacturing systems can capable of the producing tailored parts on-demand. The integration of robotic manipulators and extrusion subsystems enables the scope of additive manufacturing beyond the limitation of conventional printers, with potential domains of application including aerospace, medical implants, customized tooling, and on-site construction (Gebler, 2014).

By finding solutions to both the motion control and material deposition control issues, this work consequently advances the state-of-the-art in robotic additive manufacturing with

emphasis on real-time control, ROS 2–based integration, and hybrid hardware architectures for extrusion control.

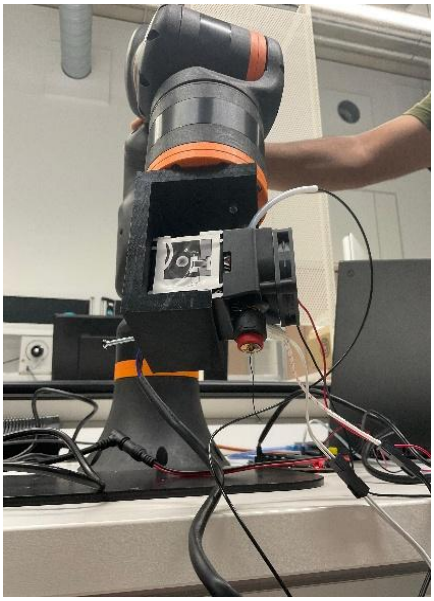


Figure 1 System overview: igus ReBeL with E3D Revo Hemera XS and SKR Mini E3 V3.0 controller; host runs ROS 2 coordinator. This figure anchors the scope of the platform used throughout the thesis.

1.2. Own Interest/Motivation

I would like to create a replicable robotic FFF workflow that other students and labs would be able to use in practice with widely available parts instead of a proprietary controller. The installation of an E3D Revo Hemera XS on an igus ReBeL and extrusion control by Marlin on a BIGTREETECH SKR Mini E3 makes the system inexpensive, customizable, and in line with how the majority of people use slicing and toolchains (Coutinho., 2023).

Technically, I am working on the synchronization issue between a streamed motion of a robot and a buffered extrusion semantics of a printer. When the motions and flow are not in phase, it causes bead-width drift and weak interlayer bonding; this then directly affects the quality of prints (Coogan & Kazmer, 2019).

Technically, ROS 2 provides an up-to-date, modular framework to model a coordinator which translates slicer G-code to two related streams of fixed-period poses to the robot and relative extrusion to Marlin taking into consideration middleware/USB jitter with bounded in-flight queues and acknowledgment-driven (ACK) flow control (Kronauer., 2021).

Lastly, I am concerned about replicability: Parameters, M-code policies, logs, and configuration diffing so that others could validate or extend the results on similar arms and extruders providing a small but reliable step toward robotic AM (Yao, 2021).

1.3. Research Gap

Recent robotic FFF projects have shown multi-axis capability, continuous spatial motions, conformal and curvature-depositing but these applications do not run standard slicer G-code end-to-end on general purpose hardware. This gap is bridged by a reproducible bridge that takes slicer output and turns it into two closely integrated streams robot poses (via CRI) and Marlin extrusion under transparent queueing and acknowledgment policies on a 6-DOF platform (Yao, 2021; Lu, 2025). The common alternatives depend on custom or specific toolchains or various forms of control structures that constrain reuse (Lu, 2025).

Guidance on timing has been developed at the middleware level (ROS/DDS), but not converted to an extrusion-synchronization controller that is tolerant of host/USB jitter. We will require a tick loop with a fixed period that translates instantaneous tool speed into extrusion, and a limited serial queue in-flight, which can only advance on Marlin ok acknowledgments, an application-level recipe not described in the literature (Paul, 2024; Kronauer, 2021; Coutinho, 2023).

Aspect	**Cartesian / Gantry FFF**	**Robot-Mounted Extrusion (6-DOF)**
Geometry handling	Planar layers, fixed tool normal	Multi-axis orientation, inclined/on-surface paths
Build volume	Limited by gantry dimensions	Larger reachable workspace
Support structures	Often heavy, time/material costly	Many supports reduced or eliminated
Process planning	Simple planner semantics	More complex toolpath planning
Part size	Limited	Scalable, extended reach
Overall trade-off	Simplicity, robustness	Flexibility, complexity

Table 1. Comparison of Cartesian/Gantry FFF and Robot-Mounted Extrusion (6-DOF)

The underlying process physics are well studied, but controller coupling is not. Bead shape and interlayer bonding depend on pressure/flow and contact conditions, which argues for

keeping motion and flow phase-aligned. A compact formulation used here is $Q(t) = w\,h\,v(t), A_f = \pi\left(d_f/2\right)^2, and\ E_{\text{rate}}(t) = Q(t)/A_f$ what is lacking are real-time implementations that embed these relations into a robot Marlin synchronization scheme with explicit timing and queue management (Coogan & Kazmer, 2019).

Available industrial grade ROS-based AM systems focus on metal WAAM, where unsafe WAAM program codes translate to safe ROS semantic process control function calls, but open-source polymer FFF systems that follow Marlin semantics in their lightweight ROS 2 integration to collaborative arms, with safety M-codes and ACK-driven flow control, are less discussed (Coutinho, 2023).

Lastly, there are not numerous replicable baselines to hybrid robot-Marlin systems. Auxiliary information public outcomes typically do not cover bead-width on demand Q, path (along the track), and error against rate, and okay-latency and queue occupancy of economically viable data controller-driven extruders (Rastegarpanah, 2024). We take a deliberately limited scope: we do not address curved-layer slicing, multi-robot/mobile AM, closed-loop flow/pressure control. The contribution is a synchronization protocol that can be reproduced on available hardware (igus ReBeL, TCP <= 0.2 m/s) that has definite safety/abortion policies and can publishable logs to allow the method to be validated/extended by others (Yao, 2021).

The initial proposal emphasized simulation-first validation using Gazebo, URDF/Xacro models, and the ros2_control framework. During execution, the scope was adapted: instead of a full simulation, the thesis validates the synchronization policy directly against a physical extruder running Marlin firmware, coordinated by a ROS 2 node. This adjustment was driven by time and hardware constraints, yet it preserved the core research objective to demonstrate that a ROS 2–based control strategy can synchronize motion execution and extrusion in real time. Simulation-based validation remains an avenue for future work.

1.4. Objectives and Goals

Provide a reproducible, end to end pipeline, to ingest a project file (.3mf) that has an embedded g-code, execute it on a 6-DOF Robot (igus ReBeL) with synchronized extrusion (SKR Mini E3 V3.0 + E3D Revo Hemera XS), driven by ROS 2 to coordinate extrusion, timing, safety, and recovery.

- **Conversion & parsing**. Unzip the .3mf (ZIP) file, depack Bambu-format gcode, and parse at least 99 percent of relevant motion/extrusion lines; non-supported codes are logged and do not cause crashing. The units are in mm, modes in absolute/relative co-ordinates, header temperatures, fan settings captured, a report is given on how many lines were interpreted, codes that were not identified, and whether any retractions occurred. Support edge cases of zero-length movement, repeated G92, comments/whitespace, travel vs extrude segmentation with an option to flatten retractions or warn when they are detected.
- **Real-time orchestration**. Run fixed-period ROS 2 loop (50 Hz goal) and stream poses and backlash relative extrusion using CRI with bounded in-flight primitives and logging of ok latencies. Employ a delayed-clock timer, small robot-command buffer, and ACK-based flow control; add serial stall and socket CRIs that cause watchdog-based safe pause/abort. The parameters of Document QoS/timer settings, in-flight bounds (N_{min}, N_{max}), timeout policy, and CPU load, and prove at most 1% failure to tick in tests lasting at least 10 minutes.
- **Safety and calibration**. Implement Marlin safety options (M104/M109/M302/M112), PID and E-steps calibration, and holding calibration constants in EEPROM. Abort/recovery: an "ok" timeout pauses the robot with limited resend; a stuck fault upgrades to M112 via the EMERGENCY_PARSER, heaters made safe and the robot parked. The diffs are the configuration.h (board, thermistor, drivers and baud), PID improvements at PLA/PETG temperatures, E-steps at a range of feed rates, fan policy, and cooldown steps.
- **Experimental verification**. Print primitive in PETG in straight line, 100 mm square, spiral, lattice sweeping speed and temperatures with 0.4 mm nozzle with 30 and 60 mm/s, and with the temperature ranging between 230 and 250 degrees Celsius. Records bead width/height at user-specified stations; measure path error (RMS/max) using overlays or vision; and measures serial timing (ack histogram, queue occupancy). One trial per condition should be conducted in randomized order and mean SD should be reported, distinguishing between corners and straight segments; provide process photos and selected cuts.
- **Reproducibility**. Publish configuration diffs, launch commands and CSV logs adequate to third-party replication. Include a well understood repository structure (data/scripts/config), system details (OS, ROS 2 version), parameter files and commit hashes and a replication checklist and a sample dataset. Include a brief

README with step-to-step instructions and an optional brief video of the run sequence.

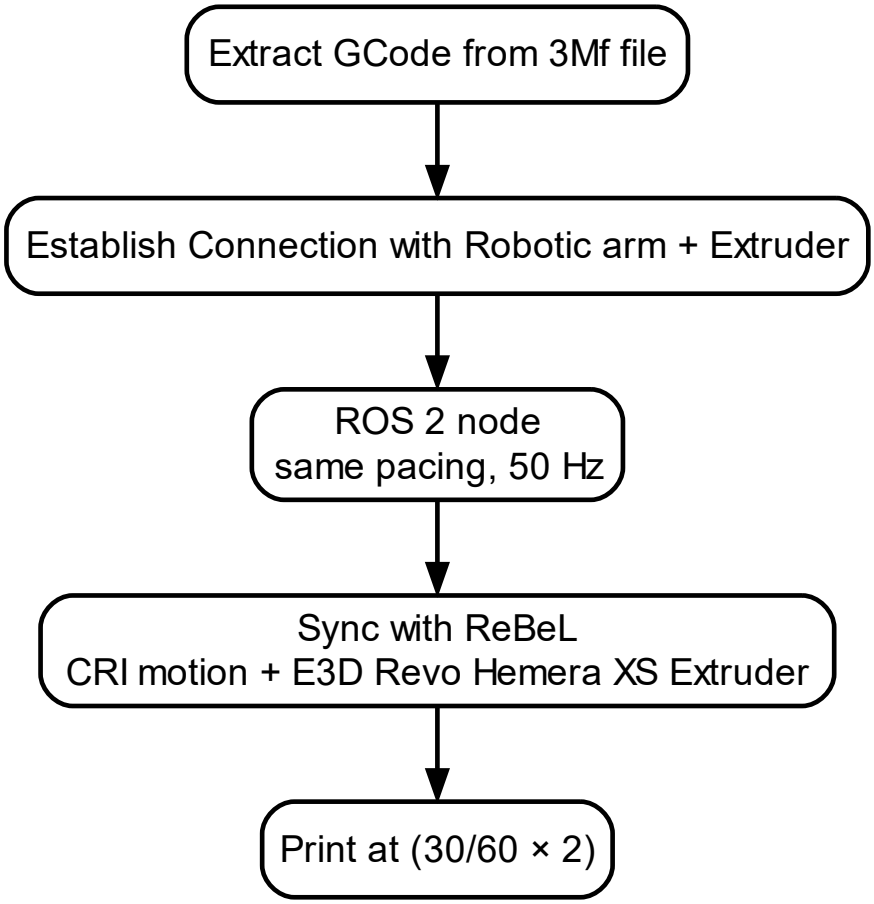


Figure 2 Path Overview

1.5. Research Questions

What synchronization settings (control period Δt and in-flight bounds) enable continuous, phase-aligned deposition when standard slicer G-code is executed on a 6-DOF robot with a Marlin-driven extruder?

What RMS/max path error is achieved up to the ReBeL TCP limit (0.2 m/s), and how do accelerations/corners affect bead uniformity?

How robust is the 3mf→.gcode→segment pipeline, and does the system fail safe under induced serial/thermal faults?

1.6. Scope & Limitations

Scope

- **Platform & materials**. Single setup: igus ReBeL 6-DOF, reachable workspace used ≈ radius 0.2–0.6 m, height ≤ 0.35 m. (TCP ≤ 0.2 m/s) + E3D Revo Hemera XS (1.75 mm) driven by Marlin on SKR Mini E3 V3.0 (USB 115200); baseline material PETG, nozzle 0.4 mm.
- **Toolpath pipeline**. Projects from Bambu .3mf → extract embedded gcode → parse a core G/M-code subset (G0/G1/G92; M104/M105/M109/M83/M302/M112; optional M200).
- **Control architecture**. ROS 2 coordinator at 50 Hz streams CRI poses and relative E ticks with bounded in-flight commands, ok-ack flow control and full telemetry logging.
- **Synchronization evidence.** One run per condition (30/60 mm/s × naïve/windowed); metrics are med/p95/p99 L_ok and in-flight occupancy N.
- **Evaluation & reproducibility**. Planar primitives (line, square, spiral, lattice); metrics: bead width vs commanded flow, straight-segment path error, ok-latency/queue occupancy; provide configs, logs, and a replication checklist.

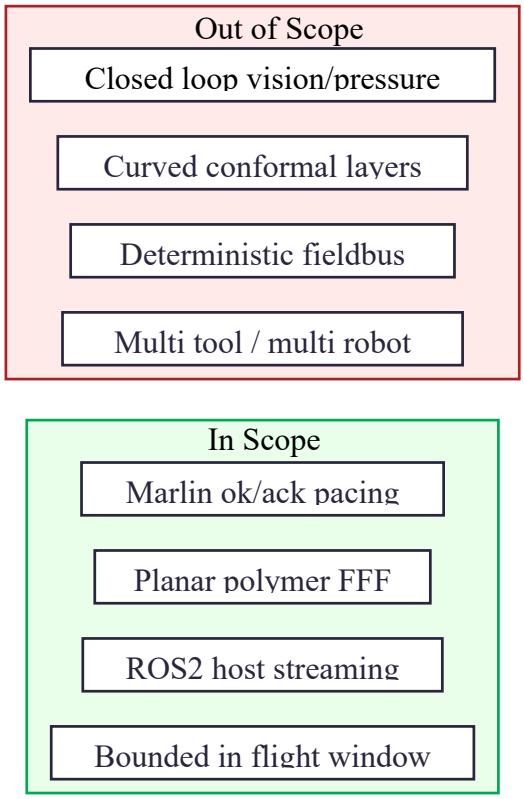


Figure 3. Thesis Scope vs. Out of Scope

Limitations

- **Geometry & bandwidth**. No continuous-layer/non-plane deposition, all arcs G2/G3 segmented; ReBeL dynamics constrained performance. Planar toolpaths only; printing on a Voron PEI build plate without bed heating.
- **Control scope**. Open-loop extrusion (no pressure/vision feedback) Linear Advance/feedforward is considered future work; no hard real-time kernel focus is feasible practical timing stability.
- **Measurement fidelity**. Path error and bead geometry measurability with readily available tools/overlays, absolute accuracy is not satisfied, no highly-precise external measuring equipment.
- **Generality & safety.** Results demonstrated on one robot–extruder–firmware stack, transfer to other platforms not validated; not an industrial safety-certified system.
- **Conformal/tilted** deposition (e.g., 12°), fieldbus transports, closed-loop bead/pressure control.

2. LITERATURE REVIEW & THEORETICAL FRAMEWORK

2.1 Introduction

This chapter summarizes research at the nexus of ROS 2 based real-time coordination and additive manufacturing (AM) in the context of continuous deposition, with the latter limited to polymer fused-filament fabrication (FFF) and design-oriented reference to its wire-arc AM (WAAM) analogue. We take the standards-conscious terminologies ISO/ASTM 52900 for AM and ISO 8373 on robotics and differentiate planner-based, buffered G-code and host-streamed motion/flow (ISO/ASTM 52900; ISO 8373). Expanding on the above presentations of the multi-axis robotic FFF inspired by the increased work volumes, tool-neutrality and the minimization of supports. we pose the key challenge in the regularizing joint tool-motion and polymer-flow in the presence of realistic middleware, transporting timing properties (Lu, 2025; Rastegarpanah, 2024).

The review is organized around five strands:

- Robotic extrusion architectures (direct-drive vs. Bowden; split robot–extruder control vs. unified);
- Robot-oriented toolpath generation (planar baselines and arc/spline handling, with conformal/curved layers noted as future extensions);
- ROS 2/DDS middleware and streaming, where end-to-end latency and jitter depend on executors, QoS, and transports (Maruyama, 2016; Kronauer, 2021);
- Firmware semantics for extrusion controllers, including Marlin's buffered execution and ok acknowledgments; and
- A Process model sufficient to reason about flow–motion coupling in FFF (Coogan & Kazmer, 2019).

We summarize measurement practices (path error, bead statistics, acknowledgment-latency distributions) and synthesize areas of consensus, disagreement, and open gaps—most notably the absence of an open, reproducible ROS 2 pipeline with an explicit synchronization policy for polymer robotic FFF (Coutinho, 2023). The chapter then concludes by conjecturing a theoretical framework that labels the pertinent constructs (e.g., control period, in-flight window, acknowledgment latency) and postulates to be tested by future methods and experiments.

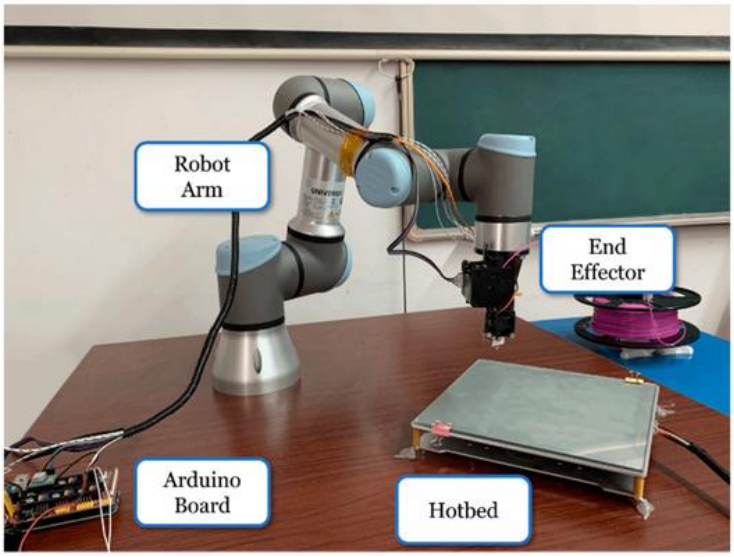


Figure 4 Embedded Robot (yaun yao, 2021)

2.2 Terminology, Scope & Boundaries

We adopt ISO/ASTM 52900 and ISO 8373 terminology to avoid ambiguity (e.g., 'planner-based G-code' vs 'host-streamed execution'). State what we simply by robotic FFF and multi-axis deposition, and explicate the distinction between the planner-centric execution of G-code and host-streamed management as shown in the literature (ISO/ASTM 52900; ISO 8373). Focus the review on polymer, robot-delivery extrusion and only refer to WAAM as an architectural analogy. Be succinct on exclusions (e.g., curved-layer printing, high-bandwidth feedback) so as not to misdirect later synthesis with in-scope illusion. Degree to justify literature window by the emergence of robotic AM framework and the FFF development maturity of ROS 2/DDS time research, with the ability to cherry-pick essence of foundational FFF mechanics

Robotic FFF / continuous deposition. Material-extrusion AM in which a nozzle mounted on a robot extrudes a continuous filament along a commanded toolpath. Polymer FFF is the most prominent area; multi-axis experiments are deployed to generate synchronization requirements. Motion or flow is emphasized on how the coordination, not necessarily in kinematics is developed (Lu, 2025).

Multi-axis deposition. Deposition with over 3 orthogonal axis (e.g. 6-DOF serial arms) so one can reorient the nozzle, enhance reachable geometry and lessen supports. The added orientation freedom increases time criticality at corners and on surface, where host/firmware coordination becomes relevant (Rastegarpanah, 2024; Lu, 2025).

Toolpath commands vs. host-streamed execution. Planner-based G-code files (precomputed motion/flow that is managed in the hardware firmware), as opposed to host-based streaming that keeps buffers full and can adjust in real time. Both the patterns and the timing implications are documented in AAM toolchains and transferred to polymer extrusion (Coutinho, 2023, IFAC).

ROS 2 / DDS middleware. Publish/subscribe with executors, timers, and QoS (reliability, history, deadlines, liveliness) shaping end-to-end latency and jitter in control loops; measured performance depends on implementation and settings, so explicit configuration matters (Paul, 2024, Edge AI meets Swarm Intelligence Workshop).

E**xtrusion firmware (planner + acknowledgments).** Embedded controllers execute a buffered G/M-code queue, regulate temperature, and signal headroom with ok. Bead quality is constrained by thermo-rheological dynamics, motivating flow models for control design (Coogan & Kazmer, 2019, Journal of Rheology).

In-flight window (introduced here; formalized in §2.11). A bounded range $[Nmin\ , Nmax\ ]$ of unacknowledged motion/flow commands allowed in downstream buffers to reduce planner starvation under host/transport jitter. We treat this as a synchronization construct, not a hardware detail.

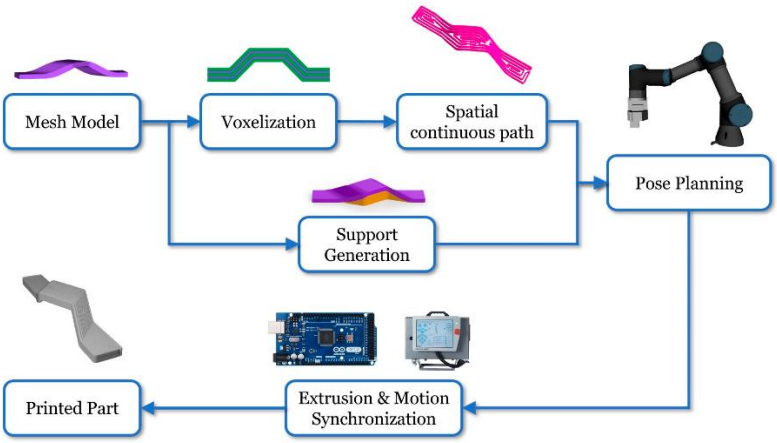


Figure 5 Printing Process with Robotic Arm (yaun yao, 2021)

Process focus. Polymer FFF with a robot-mounted extruder. The central concern is timing-correct coordination of tool-centre motion and volumetric flow along mostly planar paths a-controlled baseline for measuring synchronization effects (Lu, 2025; Rastegarpanah, 2024).

Control/middleware focus. ROS 2 executors/timers and DDS QoS as levers to bound latency and jitter in host-streamed deposition. We draw on comparative studies to motivate reproducible configuration choices rather than vendor selection (Paul, 2024).

Architectural analogue. WAAM is cited only where robot–process–feed synchronization patterns generalize (e.g., buffering, ACK-driven flow control, planner-starvation risks), sharpening the design space for polymer deposition (Coutinho, 2023).

Out-of-scope

- Curved-/conformal-layer deposition and non-extrusion AM (context only).
- High-bandwidth feedback (vision/pressure) and detailed microstructure models.
- Proprietary closed toolchains and fieldbus specifics.
- Full mobile-manipulation navigation pipelines (beyond timing implications).

Strand	What exists	What's missing	Thesis response
Architectures	Robot + process splits (WAAM/polymer)	Polymer FFF with explicit ack pacing recipe	Provide bounded-window policy
Middleware timing	ROS 2 latency studies	Settings → print quality link	Map latency tails → bead outcomes
Firmware semantics	Marlin docs/usage	End-to-end sync exemplar	Reproducible ROS 2↔Marlin pipeline
Measurement	Geometry or timing, rarely both	Unified protocol & logs	Joint timing + bead metrics set

Table 2. Gap Matrix

Literature window and inclusion criteria

Time window (2014–2025). Captures (i) the rise of robotic AM/WAAM control frameworks that expose streaming/synchronization issues and (ii) the maturation of ROS 2/DDS with public performance studies from Foxy → Humble. Earlier foundational FFF mechanics are included only where needed for minimal process models (Rastegarpanah, 2024; Paul, 2024).

Inclusion. English, peer-reviewed or persistently archived sources that report architectural patterns, timing/QoS measurements, or process models usable for synchronization design with enough methodological detail to support reproducibility (Coogan & Kazmer, 2019).

2.3 Historical Trajectory & Motivations

Robotic FFF has shifted from gantry kinematics to the multi-axis arms primarily to expand reachable geometry and reduce supports; orientation control near inclined/curved surfaces is the key benefit. These gains raise tighter timing demands on motion–flow coordination, which our work addresses with a bounded in-flight window and ack-aware pacing. To create the curved parts rapidly and with increased resolution than standard systems, they are relying on a 6R robot and a 1.0-axis positioner, as an example (Lu, 2025).

And of course it brings to anxiety the control piece. It continues to push its continuous deposition technology with relying upon smooth synchronized movement and extrusion and when the host PC or firmware falters in even a minimal amount, the jitter could spread layer lines and distort bead shapes. Flow instabilities are the result of wobble in flow. Research in WAAM typically founded on ROS and either on-form offline path control or near real-time on-line correction, renders this issue obvious: onto the once and traditionally stack of modules composing a deposition the previously consistent and exclusive presence of the robot, feedback loops, and motion planning modules. This same dynamic appears in the case of polymer extrusion where the hardware is always firmware-controlled and the robot is being host-streamed (Coutinho, 2023).

The pro-multi-axis brigade emphasizes a lower amount of support, improved bead orientation and the potential ability to perform massive or even-in-situ structures. Representing the list of demerits: the initial cost is higher, it is more complex, and poor synchronization may make advantages come to naught. The critique is supported by process studies: bead morphology and interlayer adhesion require controlled pressure and flow history because a transient in motion, such as a jump at a corner or an abrupt change of speed, manifests itself as wider joints or discontinuous layers right away (Coogan & Kazmer, 2019).

The temptation of such development path of gantry platforms into industrial arms into multi-axis robotic platforms is the geometric and scale advantages, where this advantage only comes true when the chain is in a complete lock step, namely the host scheduling of the devices, and the middle-ware transport of the device, and the firmware, and the motion

control. Recent surveys put it most aptly: the actual challenge is not just kinematics; it is developing soft- and firm-ware to support time-sensitive streaming and filling so that intended motions can be smoothly converted to non-tilting, sustained deposition.

Ref Key	Robot	Extruder	Control split	Link	Sync method	Outcome note
Yao2021	6-DOF arm	Polymer hot end	Split	TCP + USB	Host-timed (reported)	Continuous spatial paths demo
Lu2025	6-DOF + positioner	Polymer hot end	Split	TCP + USB	Host-timed	Kinematics + validation
Coutinho2023	Industrial robot	WAAM torch	Split	TCP + fieldbus	Ack/monitoring	Monitoring/control (metal)
Kronauer2021	n/a	n/a	n/a	ROS 2 stacks	n/a	Latency analysis for tuning
Paul2024	n/a	n/a	n/a	ROS 2 stacks	n/a	QoS/executor guidance
Rastegarpanah 2024	6-DOF	Polymer	Split	TCP + USB	Mixed	Process/geometry focus

Table 3. Comparative Systems (Robot, Extruder, Control Split, Link, Sync Method, Notes)

2.4 Robotic Polymer Extrusion Architectures

Collectively the choices determine the rate at which a system can respond to the latency spikes or jitter-and to the probability of planners running out during cumulative deposition. Note that similar WAAM systems (robot controller + process controller + online monitoring) can be used as a cross-domain reference in cases where the printhead itself has its own firmware and the robot is executed online (Coutinho, 2023).

Overview:

1. End-effector control (direct drive/ Bowden)
2. Split between controller (robot motion vs extrusion firmware vs unified motion-flow)
3. Connecting device (TCP driver to robot; serial/fieldbus to extruder)

4. Strategy of synchronization (planner-based or host-streamed)

End-effector. direct-drive and Bowden. Direct-drive is capable of increasing filament flow speed, resulting in better response to transient forces- yet increases mechanical mass that may starve smaller robot axes out of bandwidth. Bowden works counter to that: reducing inertia but adding line compliance and a flow-rate lag that manifests as variation of bead width, most notably when corners or acceleration/deceleration events step an extruder off equilibrium.

Controller split. The usual division of open-system robotic AM is the temperature and extrusion control and motion planning. Motion sequences are received via G-codes and performed by the robot where a distinct extrusion-specific firmware module performs coordinate syncing and flow adjustment along the E-axis. This division fast-tracks re-use of previously developed thermal logic and safety routines, but does necessitate the need to explicitly host-buffer. WAAM workflows are the example of mature split setups with mid-print adjustment of process parameters and ROS-based monitoring (Coutinho, 2023).

Communication links. Typically, robot motion is controlled using an TCP driver (e.g. CRI) and extrusion firmware is compressed using USB CDC/UART. Determinism in higher-end systems can use CAN, Ether CAT or other such protocols (CRI v17 manual). Midway, middleware investigations include measurements of timing constraints in TCP usb designs that reveal both host-OS scheduling and transport jitter that must be managed through prudent control over queue depths.

Synchronization strategies.

- **Planner-based:** a precompiled G-code stream is fed to the firmware, in which an autonomous planner controls pace and movement concurrently; host timing is secondary, but flexibility is reduced.
- **Host-streamed:** the host flows fixed-period or time-stamped commands to robot and extruder, keeping its buffers nonempty and turning over control to new commands on (its) acknowledgments; allows online adjustment of control but exposes the loop to jitter in the middleware/transport.
- **Hybrid:** robot movement is transmitted (TCP) and extrusion G-code is calculated in parallel, and cross channel buffering must be setup to prevent desynchronization of corner transitions or speed alterations.

- **Implication:** these policies of buffer-management determine the likelihood of starvation, acceptable latency tails, and attainable bead uniformity.

Buffering & flow-control patterns.

There are two strategies that reoccur across the literature are Rate-based, send with nominal cadence, with large downstream buffer. Ack-based, bound an in-flight window, and only accept an acknowledgment. Ack policies are preferable in applications where links are bursty-jitter or there is a shallow firmware buffer as is the case in printed WAAM applications (Coutinho, 2023).

- **Safety & observability** (usually an inheritance of the surrounding fields). Robotic additive stacks reveal E-stop capability, thermal protection, and process variable telemetry, queue depth and robot status. It was pointed out that there is a benefit of such metrics in diagnosing missing deadlines or a temporary bug as waves are logged by WAAM toolchains and offer web-accessible monitoring (Coutinho, 2023).
- **Common failure modes** (as well as why architectures are important). Blobs, thinning at start/stops, corner swellings and occasional gaps are regularly attributed to misaligned actuation events and buffer starvation, not just the kinematic reach. Previous flow-pressure analysis of FFF demonstrates that transient potential mismatches couple into contact pressure and bead geometry and directly connects architecture and synchronization policy to physical result (Coogan & Kazmer, 2019; Lu, 2025).

2.5 Toolpath Generation for Robots (Planar → Conformal)

When you start to study a systems-level course in 3D printing it hits you pretty soon how robotic FFF (fused filament fabrication) actually builds upon the one thing we already understand the basics of: the slicer output we already learned about, otherwise known as G-code for planar layers (G1/G2/G3 primitives) extracted directly these primitives from a CAD model. The major advantage of robots is that the end-effector (nozzle-mounted) frame is mobile around the part, and the software can map slicer motion to the frame rather than the immovable laboratory-defined coordinate system.

That convenience has one significant caveat: they must keep the tool-centre speed (TCS) close to continuous over each segment to allow a relatively steady extrusion. In transitioning slicer output to robot code, the joins and corners need to be smoothed so that the TCS profile

does not jump. The broad pipeline depicted in the papers, CAD showing limitation-to-tessellate-plus-slicing-plus-path-plan-plus-G-code, indicates that it is at the stage of reparameterization that hard constraints such as speed limits and corner-smoothing bounds creep in.

One more grand dimension of comparison involves the issue of keeping arcs or preferring a successive line segment being replaced with chords. Arcs (G2/G3) minimize command rate, since they do not have to be subjected to constant acceleration, chords (G30) tend to run denser, but can be smoother in small-chord geometries. The trade-offs between fidelity vs. controller load are not equal with each of the choices. All surveys recommend that one maintain arcs where feasible, apply arc-to-chord transition smoothing where necessary and in general limit the velocities of corners lest you prematurely over-extrude during the deceleration phase.

To add to this complexity, the robotic arms are multi-DOF meaning that introduced constraints that have to do with orientation that are absent in the stationary gantries. In this case inverse kinematics (IK) is used together with time parameterization to ensure a stable TCP velocity profile, and most investigations mention that it is not possible to divide the motion and the time separably. The coordination between the joint is critical and it has been shown that, in the majority of reports, meaningful workspace and time savings are only achieved when both coordination of nozzle- normal and coordination of timing are considered globally.

After you are able to understand the fundamentals, you may do curved or conformal layers. The good news is that they enhance both tool-normal alignment and enable the reduction or even removal of supports, but on the bad news side, they also narrow timing windows--curved paths make orientation switching quicker, orientation-sensitive velocity lag persists, and dense collision envelopes appear when going very fast. Current demos on curved surfaces emphasize the experience of the ideal chore - orographically synchronized multiaxial movement to avoid the drift of orientation and flare-ups of flows. Due to that added complexity, most teams still benchmark against planar paths and delay conformal layers until a future research effort at which time timing effects are better understood.

It is not only geometry but the toolpath is the pulse of time and clearance. Preservation or chording of arcs, invariance of joint velocities, whether trajectories are allowed to become planar or remain conformal, all affect the G-code profile, hence affecting the pressure buffer requirement, the accuracy with which the driver can deliver commands, and the maximum

firmware extrusion response time. Briefly, robotic FFF toolchains reminds us how the primitives we use and when we use them are the blind spots that disrupt Print.

2.6 Middleware & Real-Time Streaming with ROS 2/DDS

Overview. Host-streamed deposition End-to-end timing: host scheduler ROS 2 executors/timers DDS QoS and discovery transport device/firmware buffers. Latencies and jitter reported depend on DDS implementation, message size/frequency and message transport medium; and so, configuration rather than middleware selection dictates timing behaviour in practice. We consider ROS 2 / DDS as a soft real-time base, which needs to be combined with the buffering/acknowledgment policy to maintain the continuous deposition (Maruyama, Kato, & Azumi, 2016; Kronauer, 2021).

Measurement of what the literature involves. Instrumented comparative analyses of round-trip and per-frame latency with explicit timestamping and relay subscribers factor out intermediate processing to infer one-way delay; findings show nonmonotonic latency versus payload size, frequency-dependent tail effects, and intra- versus inter-vendor and same-domain versus different-domain (wired versus wireless) differences. No RMW performs the best in all cases, and bridging/multi-domain case tend to exacerbate tails- a motivation to design against bursts and outliers rather than means (Kronauer, 2021; Paul, Le Phuoc, & Hauswirth, 2024).

Patterns that count-QoS. Of collections in periodically-controlled topics, a typical design combines Reliable + Keep-Last (depth approximating (10100)) to bound memory without dropping; Deadline/Liveliness to detect stale publishers; History sizing with publisher frequency and subscriber-side processing cost. Literature points out that latency characteristics vary under QoS and payloads and thus QoS cannot be considered as a default, it should be a design variable (Maruyama, 2016; Paul, 2024).

Scripts and clocks. ROS 2 steady timers and short callbacks can mitigate the effects of scheduling spillover; multi-threaded executors can attempt to separate I/O and computational processes at the expense of introducing contention through blocking callbacks. When using an evaluation environment, the publisher frequency has a significant effect on latency with larger messages and therefore it is important to capping the volume of work that can happen during the callback, as well as maintaining low timer jitter when serving the streams in a high rate (Maruyama, 2016; Kronauer, 2021).

Buffering and control of the flow. Two patterns on the host side recur: rate-based (fixed cadence, use large buffers downstream) and ack-based (bound an in-flight window and on acks, admit new commands). In architectures which expose explicit acks/feedback (e.g., deposition monitors in WAAM frameworks), ack-driven admission is repeatedly applied to avoid starving the planner and to synchronize host pacing to device headroom. We use the term in-flight window to designate the confined interval $[Nmin, Nmax]$ of un-acknowledged commands, the construction is introduced here and defined in (2.11). (Coutinho, 2023).

Transport considerations. TCP-based robot drivers (such as documented CRI-class interfaces) and USB/UART links to device controllers are common; they are accessible but have inherent OS/network scheduling overhead, and thus can be expected to exhibit bursts and tail latencies. Deterministic fieldbuses (E.g. Ether CAT/CAN) can reduce jitter at increased integration cost; adherence to link-specific buffering and admission control is still required at application level (Commonplace Robotics, 2024; Maruyama, 2016).

The implication of continuous deposition. To do this, middleware must be set up and instrumented to (i) bound long-tail latency (QoS, executor hygiene), (ii) mitigate some residual jitter (bounded queues/history), and (iii) coordinate with device acks (in-flight window) such that the firmware/driver planner does not starve or overrun. This reflects timing-aware execution in WAAM stacks and forms a literature and reproducible foundation that would be used to develop synchronization policies in subsequent chapters (Coutinho, 2023; Kronauer, 2021).

2.7 Firmware Semantics for Extrusion (Marlin focus)

2.7.1 Blocking/blocking vs. non-blocking control of temperature.

Marlin differentiates between non-blocking temperature set (M104) where the command completes the print instantly and firmware heats in the background and blocking wait-until-at-temperature (M109) which pauses the print stream until the desired temperature is reached (heating with S, heating or cooling with R). This point is critical in host-streamed pipelines as a non-blocking M109 may idle downstream motion/extrusion queues when issued halfway through a job (documentation specifically encourages M104 to be issued asynchronously and M109 only when synchronization is necessary).

Extrusion configurations and reposition re-sets.

Firmware allows both relative extrusion (M83) and absolute extrusion (M82); another convention in streamed deposition with relative extruder control is to set the extruder position to zero at the start of the job (G92 E0) so that individual segments and layers do not drift. M83 is not affected by G90/G91 (XYZ absolute/relative) and G92 redefines the axis position the user is currently in, explicitly- semantics that slicers and streaming hosts use to maintain E-axis state cleanly.

In minimum extrusion temperature and cold-extrude policy.

Marlin implements a requirement that ground filament and cold-flow artifacts be avoided with a minimum extrusion temperature (M302), under which E-axis motions will be rejected unless specifically overridden. At host-streamed control level it is a safety interlock: E-commands received below the level point to a configuration error; on the other hand, cold-extrusion when deliberately done during calibration must be gated and logged.

Planner buffered and acknowledgments.

lines with G-/M-codes are buffered by the planner in Firmware; the host will usually deduce an assumption of the maximum buffer headroom by observing line-level acknowledgments of ok. Given that M109 prevents forwarding of queues and that the depth of planners is limited, non-ok timing host senders are susceptible to planner starvation (observable as gaps/thinning) or overrun attempts (rejections). To this end, ack-aware flow control (bounded in-flight lines) is implemented in many robotic AM stack implementations, matching the pace of hosts to the rate of acceptances by the firmware stack. See Marlin G-code references on planner semantics; documentation on ok behaviour is on command pages and meta index.

Fast abort and emergency stopping track.

M112 does an immediate shutdown (steppers and heaters off; a reset is required), and the guidance in Marlin connect fast responsiveness to the use of the EMERGENCY_PARSER, allowing an emergency command to be parsed out-of-band even when the planner is otherwise occupied. In host-streamed deposition, this offers deterministic abort route in an occurrence of serial-link congestion or motion faults.

Smoothing on the host side and Linear Advance (pressure advance).

Linear Advance (M900) software-adjusts E-motion to mitigate nozzle pressure lag caused by the changing print speed, and can decrease corner bulging and infill artifacts, when adjusted correctly, among other things. Since LA varies the instantaneous E-profile within acceleration areas, host-side E-rate smoothing can work only conservatively, to avoid interference with firmware-based compensation.

Cmd	Intended to	Blocking?	Risk	Mitigation/Policy
M104	Set temp (non-blocking)	No	Lag to setpoint	Issue early, monitor temp
M109	Wait to temp	Yes	Planner stall mid-job	Use only pre-print/pause
M83	Relative extrusion	No	Mode drift	Set once at job start
G92 E0	Reset E position	No	Mis-zero if mid-job	Do once at job start
M302	Min. extrude temp	No	Cold extrusion blocked	Configure properly, don't override
M112	Emergency stop	Immediate	Hard abort	Log last-N, safe cool-down
M106/M107	Fan on/off	No	Cooling artifacts	Fix policy, log changes

Table 4. Firmware Command Semantics

2.8 Process Model for Continuous Deposition

During the development of the system for polymer FFF, we have arrived at set of equations that allow us to reason about timing coupling flow and temperature effects without committing to a full thermo-rheological simulation. The equations relate commanded tool speed to volumetric flow and tool feed rate and show where the difference between real materials and model materials appears due to pressure and thermal delays. An output is a control-side model that is consistent with previous process investigations that revealed that bead geometry and bonding are sensitive to pressure and temperature history, particularly to start, stop, and corners.

2.8.1 **Low levels of numbers and associations**.

Let w = width of the line [mm], h = the height of the layer [mm], $v(t)$ = the speed of the tool-centre [mm/s], df = filament diameter [mm], $Af = \pi(df/2)^2$ $[mm^2]$ The volumetric flow and the filament feed rate will be:

$$Q(t) = whv(t)\ [mm^3/s], \qquad E(t) = \frac{Q(t)}{A_f}[mm/s]$$

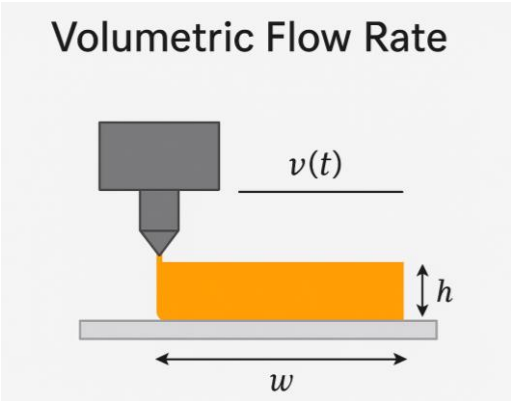


Figure 6. Volumetric Flow Rate Q(t) (created with assistance from AI software)

Such relations correspond to canonical FFF reasoning, and they present the control-side: they define the mapping between kinematic targets and extrusion. In the case of multi-axis trajectories, the maintenance of (almost) constant TCP speed assists in stabilizing $Q(t)$A theme in robotic FFF case studies Q(t).

Increments and discrete streaming. At a fixed period with host-streamed control Δt We add in small increments of filament : $\Delta Ek = E^{\cdot}(tk)\Delta t, E \leftarrow E + \Delta Ek$. It is the node in this discrete approximation that does synchronized motion/flow: it allows the host itself to make small relative E-moves in synchrony with a motion control period, with planners in devices picking up any residual jitter due to buffering out, as long as these buffers do not become starved. Buffer starvation and timing transients are the most witnessed failure modes reported in deposition studies.)

2.8.2 **Policies and constraints (why the model is lean?).**

Constant filament diameter (no ovality); (2) open-loop extrusion (no melt-pressure feedback); (3) planar toolpaths (timing tests only); (4) nozzle/bed setpoints (at fixed values, within tolerance of reality). These assumptions decouple the synchronization issue; more

detailed models (heating/cooling, viscous compressibility, wetting) are not assumed away but not needed to design the host buffering and admission policy.

Pressure and temperature dynamics introduce lags at starts, stops, and corners; these transients affect bead shape and interlayer contact. We therefore treat corners/large Δv as high-risk regions and apply mild, bounded smoothing of E-increments, while ensuring buffers never deplete. Firmware-level pressure compensation (Linear Advance) is compatible so long as host shaping remains conservative.

Connect to the future constructions. we would formalize the in-flight window $[Nmin\ , Nmax\ ]$ and the latency in acknowledgment LOk as timing variables which control the occurrence of gaps in the receipt of the discrete stream above by the extruder planner. The S-flat equations provided here give the $(t) \rightarrow E$ The mapping that that timing model employed.

2.9 Measurement & Benchmarking Practices

In the case of evaluation breakdown, you can follow four major strands, which are geometry, throughput, thermal/process dynamics, and middleware/ transport timing.

2.9.1 Bead metrology & geometry

First of all, FFF printing based on polymers uses cross-sections to connect geometry and melt pressure/temperature. The procedure is easy: lay regular lines or pad and cut in top-down and calculate bead width $\bar{w} \pm \sigma w$ and height $\bar{h} \pm \sigma h$ via fixed windows. you may choose to correlate bead size against pressure traces, as Coogan-Kazmer did with nozzle pressure against interlayer contact and geometry. That connection is useful even in a situation where pressure logging is not possible.

Path error: - Second, compute Cartesian tracking error between commanded TCP trajectories and actual path. Compute root-mean-square (RMS) error and maximum error by segment (straight lines, squares, spirals, lattices). WAAM case studies graph simultaneous position error with process variables, and we are borrowing that format here.

Throughput & stability (host streaming device)

To monitor the buffer occupancy: throughput, the number of unacknowledged TCP commands $N(t)$ impose restrictions [$N(t)_{min}$, N_{max}]. Since acks are unreliable concentrate on percentiles and starvation episodes $N(t) \rightarrow 0$

Ack/ok delay: - Each command sends and its acknowledgment time-stamped. Report the medians, 95th and 99th percentile latencies, and tail indices in order to indicate excessive delay.

Queue depth: - In case of exposure to buffer depth, record this explicitly, in case not deduce through pacing. WAAM frameworks are built on synchronized, time-stamped logs (ROS log files or "rosbag" records so we adopt the same approach.

2.9.2 In- between timing/transport timing

Describe the ROS 2/DDS pipeline. To measure transport latencies, count on a ping method, with each RMW message as one packet. Determine return time RTT=$T2-T1$ and Flip Turn. $T4-T3$ one-way latency, $L = RTT - \frac{(T4-T3)}{2}$. On frequency and size of messages, performance caches on such parameters. Compare latencies over dielectric media, wired Ethernet and USB may differ, and plot how such tails vary with message size and inter-message interval. Loss of packets under heavy load is important, therefore, be sure to monitor host CPU load just as you do in VM-based testing.

The device/driver heartbeat

When diagnostic logging is configured, do not overlap firmware ALIVE time slot. As an example, the CRI manual requires a 20-50ms ALIVE period in jogging, thus sample diagnostics must be at least 20 ms. Infer ack pacing when heartbeat interval takes more than 1 s.

2.9.3 Thermal/ process lines

Record nozzle/bed hot end temperatures and any flow surrogates that are present. In the absence of a pressure sensor, tag start/stall/corner, pressure transients at these points correlate to bead shape and height, Coogan Kazmer. Reporting conventions (make it replicable). Publish SD, the number of trials and confidence limits where relevant. Upload complete geometry descriptions, material/nozzle configuration, ROS 2 QoS/executor settings and print trajectories.

The pair width/height cross-sections are correlated with synchronized traces of speed, ack latency, and buffer occupancy. Time Cost produce a timing to cover implementation, concepts of rmw, QoS profiles, message sizes, frequencies, media transport (USB, serial bridges, wired-Ethernet), and system CPU load; literature points out the degree to which these variables influence latency. Using these guidelines, you will be able to populate a powerful toolset of metrics, including geometry, path error, throughput/stability, thermal/process dynamics, and middleware/transport timing, which supports monitoring-focused AM stacks such as WAAM.

2.10Synthesis: Agreements, Disagreements, and Gaps

Converging Findings: Agreements.

Multi-axis geometry and reachable envelopes grow, although coordinated motion and flow timing is essential; only through coherent, coherent–cooperation can the complete set of advantages come to the fore (Lu, 2025; Rastegarpanah, 2024).

It also needs motion buffering, pacing with acknowledgments: visible errors must depend on tails, not averages, and tail control is critical (Kronauer, 2021; Paul, 2024).

Non-geometric complexities are not seen on simple planar paths that can be used to demonstrate synchronization performance and expose queueing problems (Rastegarpanah, 2024; Lu, 2025).

Bead quality is transient-sensitive, and start/stop and corner timing is directly connected to bead width and interlayer contact; thus, it is important to coordinate motion and flow (Coogan & Kazmer, 2019).

Active Disputes (Disagreements).

What is the place of synchronization? Certain companies feed G-code into the planner allowing firmware to control movement and flow, others stream host-timed with bounded buffer capacity and ack-based pace control; hybrids also emerge, and empirical comparisons are context specific (Coutinho, 2023).

Pressure compensating point. Linear Advance at the firmware level can mitigate corner overshoot, although the interactions of firmware and host-side E-rate

compensation" and conservative host shaping with LA active (Marlin Firmware Docs; Coogan & Kazmer, 2019).

Arc retention as opposed to chordal zoning. Retaining G2/G3 lowers the command rate and buffer pressure but most robotics stacks continue to chord arcs, hence remaining jitter-sensitive; reported preferences vary with drivers and controller support (Rastegarpanah, 2024).

What Is Not Quite There (Gaps).

Reproducible pipeline ROS 2 open, clear sync policy. Not many polymers robotic FFF papers adequately demonstrate QoS/executor selections and queue upper limits and ack policies, and the measured latency distributions in a manner that could be reproduced (Kronauer, 2021; Paul, 2024).

Determined timing tails linked to beads performance at the working velocities. Timing or geometry may be reported in research; systematic plots of ack-latency percent or in-flight occupancy as a function of bead width or path error at modest TCP rates are rare (Coogan & Kazmer, 2019).

USB viability/portability to fieldbus. Deterministic networks are assumed in many, but a lightweight recipe (but that is usable over USB CDC (with appropriate buffering and acks) and could scale to fieldbus is under-documented (Coutinho, 2023).

Standard slicer G-code ->firmware extruder, robot, from end to end. Slicer semantics-to-robot streaming and firmware acks are clear, documented, cross-commodity examples (no indent ROS 2, robot driver, Marlin-class controller) that are, instead, unusual (Coutinho, 2023).

2.11 Theoretical Framework (Constructs, Assumptions, Propositions)

We model how host-streamed timing affects deposition by linking the control period (Δt = 20 ms) and the in-flight window ([8, 12]) to bead width/continuity and abort behavior (i) timing/queueing in host-streamed execution to (ii) deposition outcomes (bead geometry, continuity) and (iii) safety behaviour. These variables map directly to : we test whether [8, 12] prevents starvation (N=0), how median/p95 L_{ok} tracks TCP=30/60 mm/s, and how

timing tails align with bead uniformity on the square coupon. (Maruyama, 2016; Kronauer, 2021; Coogan & Kazmer, 2019).

2.11.1 Constructs & notation

Kinematics/flow. Let w [mm] be line width, h [mm] layer height, $v(t)$ [mm/s] TCP speed, df [mm] filament diameter, and $Af = \pi\left(\frac{df}{2}\right)^2 [mm^2]$. $Q(t) = whv(t)\left[\frac{mm^3}{s}\right]$, $E(t) = \frac{Q(t)}{Af}\left[\frac{mm}{s}\right]$.

Host streaming (periodic coordinator). Host emits commands at fixed period Δt (target frequency fs=1/Δt, for relative extrusion, the per-tick increment is $\Delta Ek = E^{\cdot}(tk)\Delta t, E \leftarrow E + \Delta Ek$.

Acknowledgment latency & in-flight window.

Let $\boldsymbol{Lok}$ be the random variable for acknowledgment (ok) latency from the device/firmware for a line admitted at time t. The host bounds the number of unacknowledged commands $N(t)$ within an in-flight window $[\boldsymbol{Nmin}, \boldsymbol{Nmax}]$ Starvation occurs when $N(t)$ = 0 while the planner still needs data.

Queueing identity (rule-of-thumb).

Under steady send rate fs and stationary $L\,ok$, the expected in-flight occupancy satisfies $E[N] \approx fsE[Lok]$. To bound starvation with high probability, $Nmin \gtrsim \left\lceil\frac{quantilep(Lok)}{\Delta t}\right\rceil$, $p \in [0.90,0.99]$, and choose $Nmax \geq Nmin + m$ (small margin m) while remaining below firmware/driver buffer limits (Kronauer, 2021).

Continuity indicator. Define $S(t) = 1[N(t) = 0]$ A healthy stream keeps $\mathbf{P\,r}[\boldsymbol{S(t)} = \mathbf{1}]$ near zero and limits burst lengths of $S(t)$ when they occur.

Geometry/error metrics (used later). Bead-width error $\varepsilon w = \frac{\{w_{\{\{meas\}\}} - w_{\{\{tgt\}\}}\}}{\{w_{\{\{tgt\}\}}\}}$, with $w_{\{\{tgt\}\}}$ from the flow relation above; path error as RMS/max of Cartesian deviation along the commanded path; acknowledgment timing summarized by median/95th/99th of Lok

Safety state (abstract). States {IDLE, HEAT, PRINT, ABORT, FAULT} with transitions guarded by firmware semantics (e.g., M104/M109, M302, M112) and host watchdogs; safety analysis records time-to-halt and last-N TX/RX (Marlin docs; iWAAM patterns).

Assumptions (baseline) Open-loop extrusion (no melt-pressure or vision feedback); 2) Planar toolpaths for evaluation baseline; 3) Stable setpoints (nozzle/bed within tolerance); 4) Stationary timing during a print segment (statistics of Lok stable over windows); 5) Bounded downstream buffers (robot driver & firmware planners have finite depth); 6) Constant filament diameter at the scale of interest. These isolate synchronization effects before considering richer models or feedback (Coogan & Kazmer, 2019).

2.11.2 **Propositions (mapped to research questions)**

P1 — Bounded in-flight windows suppress starvation under jitter. Given a soft real-time stack (ROS 2/DDS) with non-negligible jitter, enforcing $[Nmin, Nmax] with\ Nmi\,n \geq \lceil quantilep(L\,ok)/\Delta t \rceil$ reduces $\mathbf{Pr}\,[\boldsymbol{S(t)} = \mathbf{1}]$ and the length of starvation bursts relative to unbounded or purely rate-based sending.

P2 — In a mid-range process window, bead width tracks commanded flow. On straight, constant-speed segments with adequate heating, $|\varepsilon w|$ remains within a small band (e.g., ≤10%) when the stream does not starve; deviations correlate with timing transients and temperature excursions. Tests: print line coupons at several velocity. Maps to: FRQ2 (flow → geometry link). (Rationale: bead geometry and interlayer contact depend on flow/pressure history; stable timing yields stable geometry.) (Coogan & Kazmer, 2019).

P3 -Corner transients and flow smoothing. When the tool slows or turns at corners, the extrusion flow and nozzle pressure often lag, leading to local over-deposition (corner bulging). Minuscule host-side smoothing would suppress such artifacts, as long as the command buffers are not allowed to get empty. Test concept: sheet square path print, before and after smoothing, swelling of corners and buffer starvation measures. Maps to: FRQ3 (corner behaviour). (Rationale: deposition uniformity at transients depends on balancing smoothing with continuous command flow.) (Coogan & Kazmer, 2019; Marlin LA docs).

P4 -Ros 2 executor and timer selection. Latency distribution is dependent on different executor/ timer combinations within ROS 2. Reducing long-tail delays (worst-case response times) lowers the risk of buffer starvation and improves print continuity, even if average

latency is unchanged. Test idea: perform A/B test on alternative executor, timer and QoS settings; compare the instances of starvation and the latency characteristics. Maps to: FRQ5 (middleware choices). (Rationale: starvation risk is shaped by rare, long delays, not just mean latency.) (Kronauer, 2021; Paul, 2024).

P5 -Safety handling of blocking firmware. Blocking commands such as M109 (wait for temperature) can stall the planner if issued mid-job, while emergency commands (M112) must always be parsed quickly. Firm handling consists of bottling up the use of blocking commands during set-up scenarios and to maintain responsiveness of emergency paths. Test idea: create fault simulated by blocking commands and find the response of the emergency stops and integrity of log. Maps to: FRQ4 (pipeline robustness & safety). (Rationale: firmware semantics strongly influence pipeline stability and fault recovery.) (Marlin docs; Coutinho, 2023).

2.12 Positioning of This Thesis

The present thesis lies in the line of robotized polymer FFF and streaming based on ROS 2/DDS, where the problem of synchronization is still described in the literature but has not previously been realized in reproducible and full end-to-end environments. Specifically, we contribute: (i) a host-streamed control policy grounded in queueing/timing constructs (bounded in-flight window, acknowledgment-aware admission) rather than ad-hoc rate sending; (ii) a measurement protocol that links middleware timing—median and tail of acknowledgment latency—to bead geometry and continuity on canonical path primitives; and (iii) a transparent reporting template (QoS/executor profiles, send period, buffer bounds, timing traces) enabling independent replication.

Compared to previous work, we use the planner-vs-host synchronization argument as a design axis, and we give an argument summarizing under which conditions host timing and bounded-buffer resources are adequate to maintain continuous deposition on available transports, whether or not they result in portability to deterministic links. WAAM frameworks motivate our architectural stance (controller + process device + online monitoring), but our scope and metrics are tailored to polymer FFF. Theoretical propositions in §2.11 operationalize these commitments and pre-register the expected effects of buffering and timing on deposition outcomes (Coutinho, 2023; Coogan & Kazmer, 2019; Kronauer, 2021; Paul, 2024; Maruyama, 2016).

2.13 Summary

Chapter 2 established a standards-aligned vocabulary (AM/robotics), surveyed architectures (end-effector, control splits, links, synchronization), toolpath generation (planar baselines, arc handling, conformal prospects), middleware timing (ROS 2/DDS executors, QoS, transport effects), firmware semantics (blocking vs non-blocking temperature control, planner/acks, safety), and a process model sufficient to reason about flow–motion coupling. We compiled measurement practices for geometry, throughput/stability, timing, and thermal/process channels, and synthesized agreements (multi-axis gains require coordination; tails dominate) and disagreements (synchronization locus; arc preservation).

The chapter concludes with an outline of a theoretical framework ta constructs, assumptions, and propositions which directly align with our research questions and with our experimental design. Chapter 3 specifies the system architecture without ambiguity; Chapter 4 details the methods that instantiate the synchronization policy (bounded in-flight window, ack-aware admission, timer/executor hygiene) and the logging/analysis used to test.

3. SYSTEM ARCHITECTURE

3.1. Design Objectives and Constraints

The system is designed to run slicer generated tool paths on a six degree of freedom (6-DOF) arm and coordinate material extrusion with robot motion using commercially available and freely defined components. It is designed with three principles; host-timed streaming with explicit pacing (formalized in Chapter 4), split control, robot motion via a TCP client of the CRI interface and extrusion via Marlin over USB serial, and transparent telemetry, i.e., both links publish acknowledgments or status available to admission control and monitoring. Such decisions are influenced by practical limitations: the working space of the igus ReBeL and the maximum nominal TCP speed of 0.2 m/s 1 11 (as listed), finite buffer depths of both the robot hardware and the extrusion firmware, and the serial bandwidth of the extruder controller itself (115200 baud). The CRI server demands a keep-alive (ALIVEJOG) message and listens on a predefined port; under embedded operation the default port is 3920, and the connection is terminated once no alive message is heard after about 1 s (recommended alive update times are 20-50ms during jogging, 200-500ms otherwise).

The architecture is designed first with the principle of reproducibility; any details of communication parameters, different modes of firmware and mechanical interfaces are published in the Implementation chapter and provided in this chapter in table form. These data paths, wiring, and network topology underlying the methodology are revealed in compact form. While the proposal originally planned a complete URDF/Gazebo environment, the final implementation uses the physical IGUS ReBeL arm and Marlin-based extruder to demonstrate the same coordination architecture in practice.

3.2. Hardware Architecture

Controller and robot. The core manipulator is an igus ReBeL 6-DOF arm with embedded controller that outputs the CRI protocol via TCP. The controller broadcasts periodically such status information (e.g., STATUS, RUNSTATE), accepts client commands that are framed as human-readable messages; such connections may be shared by multiple clients, but only one sending client at a time to prevent conflicting with the same command.

The cell consists of a bench-mounted IGUS ReBeL, a Voron build plate as the substrate, and an E3D Revo Hemera XS extruder controlled by a SKR Mini E3 V3.0 (Marlin). The robot is fixed to the bench but may traverse a cylindrical workspace of approximately **r = 0.2–0.6**

m and **z ≤ 0.35 m**. In this study we exercised planar paths within that envelope; photos of planar coupons are provided as qualitative evidence.

End effector: Extruder and Hot-end. A lightweight E3D Revo Hemera XS (1.75 mm filament) direct-drive extruder (Figure 7) is placed at the tool flange with 0.40 mm nozzle. The assembly is supported on a bespoke C-section mount (3D printed in the university lab), which mounts the ReBeL flange to the body of the Hemera. The C-section offers (i) a level datum against which to define nozzle/TCP, (ii) rigidity against deposition load, and (iii) shielding of the heater, thermistor, and fan harness. Heatsink fans and part-cooling are installed and airflow is not blocked. The end-effector mass and envelope are noted to check the wrist limits.

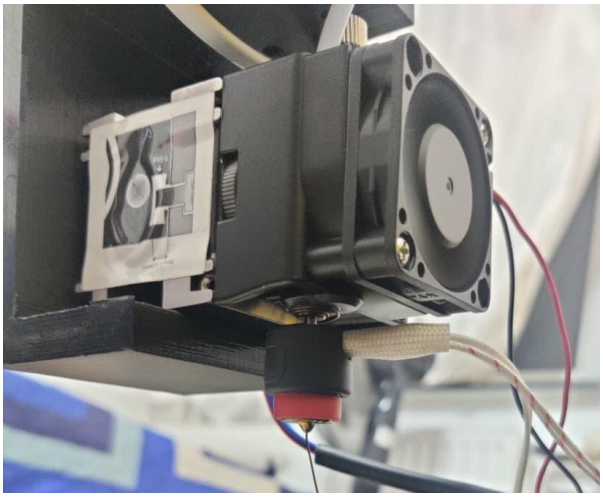


Figure 7. E3D Revo Hemera XS at the tool tip

Extrusion controller. On-arm BIGTREETECH SKR Mini E3 V3.0 (Figure 8) with Marlin drives the E-axis stepper and hot end temperature loop. The board is connected to the host by USB CDC at 115200 baud. Later firmware modes include relative extrusion (M83), position reset (G92 E0), non-blocking temperature set (M104), optionally blocking preheat (M109) restricted to pre-print stages, minimum extrusion temperature (M302), emergency stop (M112); actual responses are returned as ok lines.

Figure 8. BIGTREETECH SKR Mini E3 V3.0

Accessories and fittings. The arm is clamped to a machined base plate with M6 bolts at the manufacturer bolt circle; other clamps are used as necessary to fix the base plate to the bench. The controller is connected to the host network with the help of a shielded LAN cable. External mushroom E-stop device is placed within reach of operator and connected to break controller enable path / mains feed to PSU (per lab safety practice). C-section extruder mount is bonded to the tool flange using countersunk fasteners; cable strain-relief is placed around joints 56, to eliminate fatigue. The material and dimensions of the flat build surface are rigidly fixtured and used as the planar reference; external-validity check in Chapter 6.

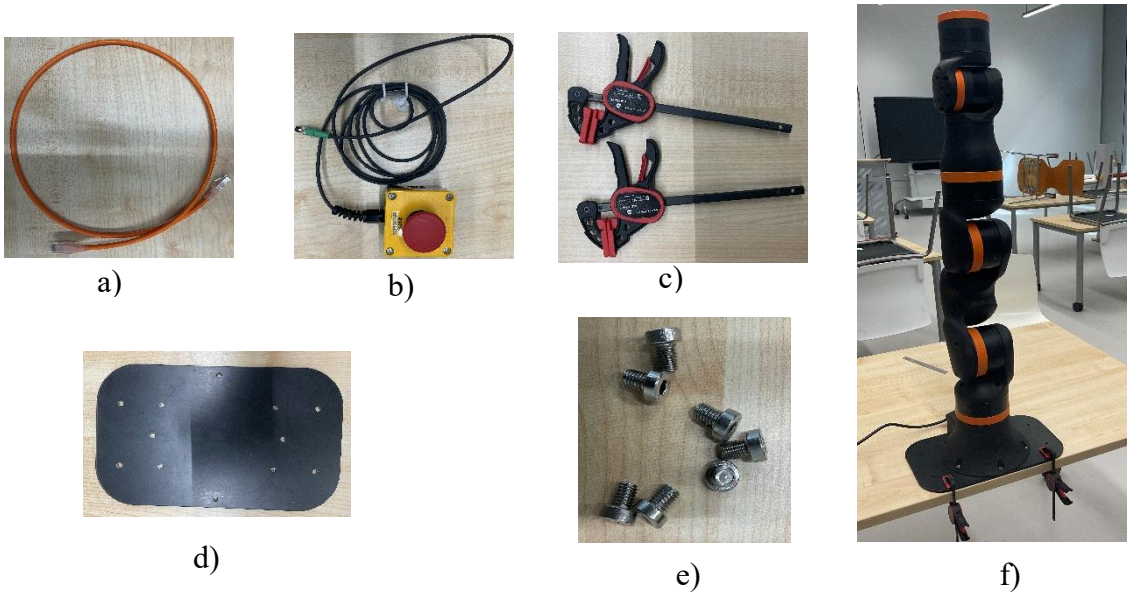


Figure 9. Fixturing & Safety: (a)LAN Wire, (b) external emergency-stop, (c) Clamps, (d) Base Plate, (e) M6 Bolts for Base Plate, (f) Fixed igus Robotic 6Dof arm.

Product Name	Supplier	Qty	Unit Price
E3D Revo Hemera XS – Lightweight Direct Drive Extruder	3DJake	1	149€
BIGTREETECH SKR Mini E3 V3.0 – 3D Printer Controller Board	Roboter-Bausatz.de	1	69€
24V Power Supply Unit + Wiring Kit	Amazon	1	34€
		Total	252€

Table 5. Bill of materials and key spec

3.3. Electrical Integration and Wiring

Board mapping. The SKR Mini E3 V3.0 is electrically configured as: HE0 to the hot end heater cartridge; T0 to the hot end thermistor (sensor type define in firmware); E0 stepper driver to the extruder motor (TMC2209); FAN0 to the heatsink fan (always-on or thermostatic per configuration); and FAN1 to the part-cooling fan (software-controlled via M106/M107). Unused axes and end-stop are deactivated to prevent false alarms; the reassignment of any I/O should be elaborate in the Table 6.

Routing and harnessing. Heater, thermistor, fans, and stepper pairs are run in one tool-side harness along the forearm. At 5-6th joint, cables are sleeved, strain-relieved (minimum bend radius is 10x the diameter of the cable) and provided with enough slack to allow articulation at the wrists. USB cable to the SKR is shielded and separated from heater lines; a ferrite bead was placed on the controller side to decrease conducted noise. A star configuration in the PSU negative; sensor returns are common to thermistor reference to reduce offsets. There is quick-disconnect around the tool flange to allow mount changes and maintenance.

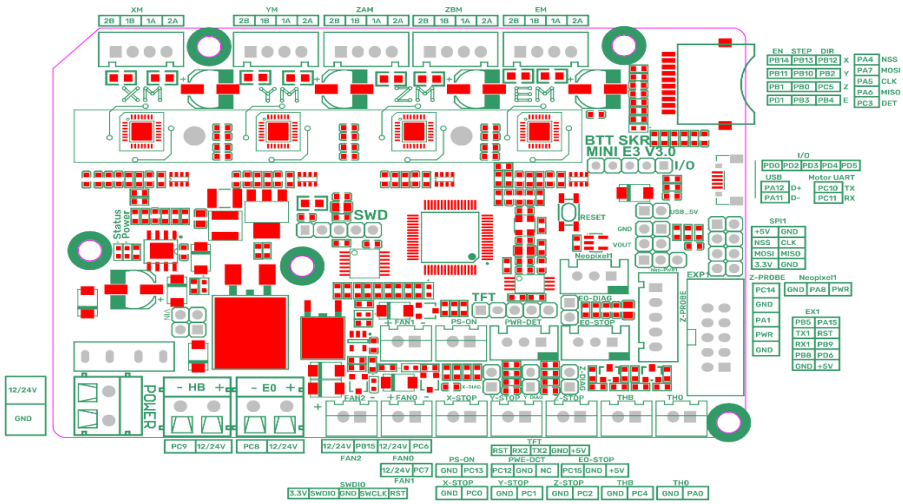


Figure 10. Schematic Diagram of SKR Board

with labelled headers: motor drivers, heater and thermistor terminals, fan headers, end stop, power input, USB, and auxiliary I/O.

Port	Device	Connector/Cable	Notes
HE0	Heater cartridge	2-pin	—
TH0	Thermistor	2-pin	Type = TEMP_SENSOR_0 <5>
E0	Extruder stepper	4-pin	TMC2209

FAN0	Part cooling fan	2-pin	M106/M107
FAN1	Heatsink fan	2-pin	Auto/manual
USB	Host PC	USB-C/-B	CDC 115200

Table 6. SKR Connection Map

Force and immunity. Power budget (Table 7) is used to size the PSU with a minimum 30 % headroom, and both the main rail and (where specified) a dedicated fan rail is fused. The controller-enable or PSU line is interrupted by the external E-stop to provide a fail-safe de-energizing. Thermal runaway protection is still on in firmware; the minimum extrusion temperature is applied to disable cold extrusion.

Load	**Voltage**	**Current (A)**	**Power (W)**	**Notes**
Hot end heater	24 V	1.7	40	Example
Extruder stepper	24 V	0.85 (avg)	—	Driver limit
Fans (total)	24 V	—	—	—
Board logic	5/24 V	—	—	—
TOTAL			PSU ≥ (sum × 1.3)	Fuse ratings: main, fan rail

Table 7. Power budget and fusing

3.4. Communication Links and Network Topology

TCP/CRI robot link. The server opens a TCP connection to CRI server on the embedded controller. The standard port value in embedded mode is 3920; it is set in server mode to expect periodic normal ALIVEJOG messages and may close the link after not receiving any in =1 s (recommended alive periods: 20 ms - 50ms in jogging). STATUS and RUNSTATE are emitted periodically by the server; units are millimetres and degrees; client/server message frames follow the CRISTART - Independent multiple -Counter CRIEND scheme. The server can support many clients, but only one active sender simultaneously (multiple passive clients may observe state).

USB/E3D/EEPROM/extruder connector (USB/Marlin). Dual G/M-code lines are transmitted to the SKR/Marlin controller at 115200 baud over a USB CDC serial connection to the SKR/Marlin board. The firmware recognizes the accepted lines with ok. Non-blocking temperature instructions (M104) are issued during the deposition process; use of blocking

heat-and-wait (M109) is limited to pre-print operations or determination-based delays. Relative extrusion (M83) is claimed just once at job start and E is zeroed using G92 E0.

Topology and timing. Both connections are terminated on the host PC and all transmissions (acknowledgments) are time-stamped to enable synchronized logging. The two-host option in the transport validity test (Chapter 6) puts an additional relay node on the second current machine linked via wired Ethernet; the CRI and USB endpoints continue as before.

Link	Endpoint	Param	Value	Notes
CRI/TCP	Robot controller	Port	3920	Alive required
CRI/TCP	Host → Robot	Alive interval	20–50 ms (jog)	Timeout ≈1 s
USB/Marlin	SKR board	Baud	115200	CDC
USB/Marlin	Ack token	"ok"	Newline-terminated	

Table 8. Communication settings

3.5. Software Stack (High-Level)

The host carries a minimal ROS 2 stack, which leaves the control of timing in order to expose all signals necessary to do logging and analysis. There are four logical parts employed. It is the responsibility of the coordinator to produce fixed-period motion updates and synchronized extrusion increments and to maintain the acknowledgment-aware pacing window. The CRI client keeps station with the TCP connection to the robot controller, sends motion instructions with limited frequency and listens to status updates to monitor health. The Marlin serial driver streams G/M-codes to the SKR Mini E3 V3.0 via USB CDC, and decodes ok confirmation, and exposes buffer headroom to the coordinator. A logger records time-stamps of all transmissions and acknowledgments on each link and maintains synchronized records along with temperature and commanded speed readings. The slicer provides an output which a lightweight G-code front end translates to time-parametrized segments and maintains arc primitives as supported.

Middleware. All experiments were run on ROS 2 Humble with CycloneDDS (RMW_IMPLEMENTATION=rmw_cyclonedds_cpp). The node used the default single-threaded executor. Publishers employed RELIABLE reliability, KEEP_LAST(depth=10) history, and VOLATILE durability. Timer period was 20 ms (50 Hz).

The stack uses ROS 2 publishers/subscribers just as a scheduling and data flow system, and deposition is not out-sourced to more senior planners. The resulting topology does not have the hidden buffering that is difficult to see and maintains explicit visibility on the depth of queues, pacing choices and timing distributions. In Figure 11 ROS 2 node graph showing /rebel_marlin_coordinator publishing /coordinator/heartbeat (liveness) and /coordinator/inflight (planner occupancy). Evidence that ROS 2 is the orchestrator in the final system. In Figure 12, Terminal ros 2 topic echo for /coordinator/heartbeat (boolean) and /coordinator/inflight (integer). Live values show occupancy clustered around 9–10, consistent with the configured window.

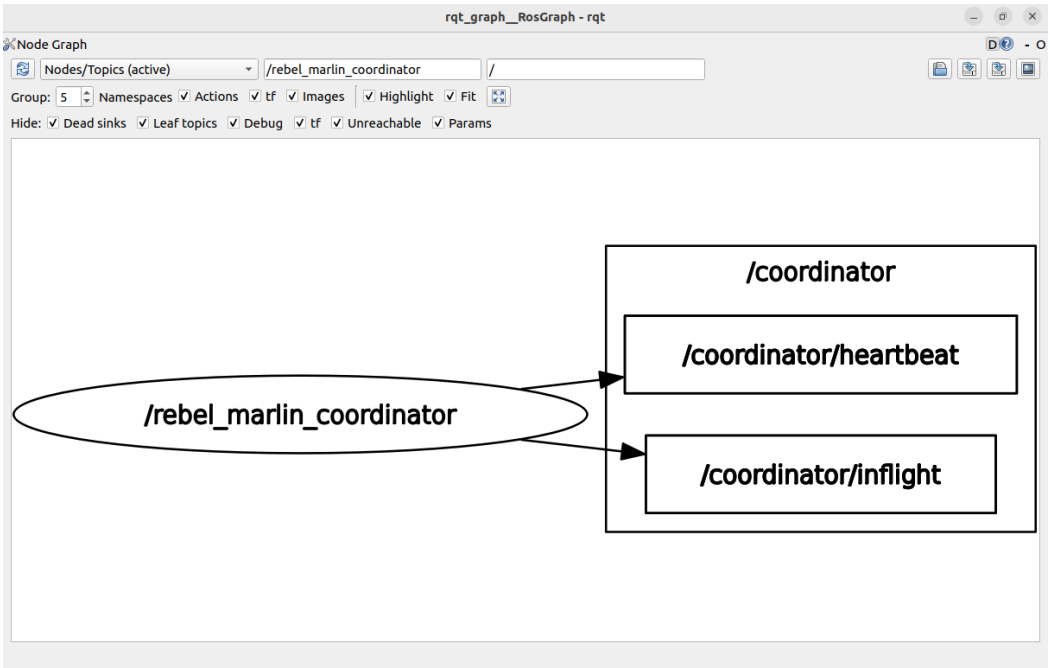


Figure 11 ROS 2 Topology (rqt_graph)

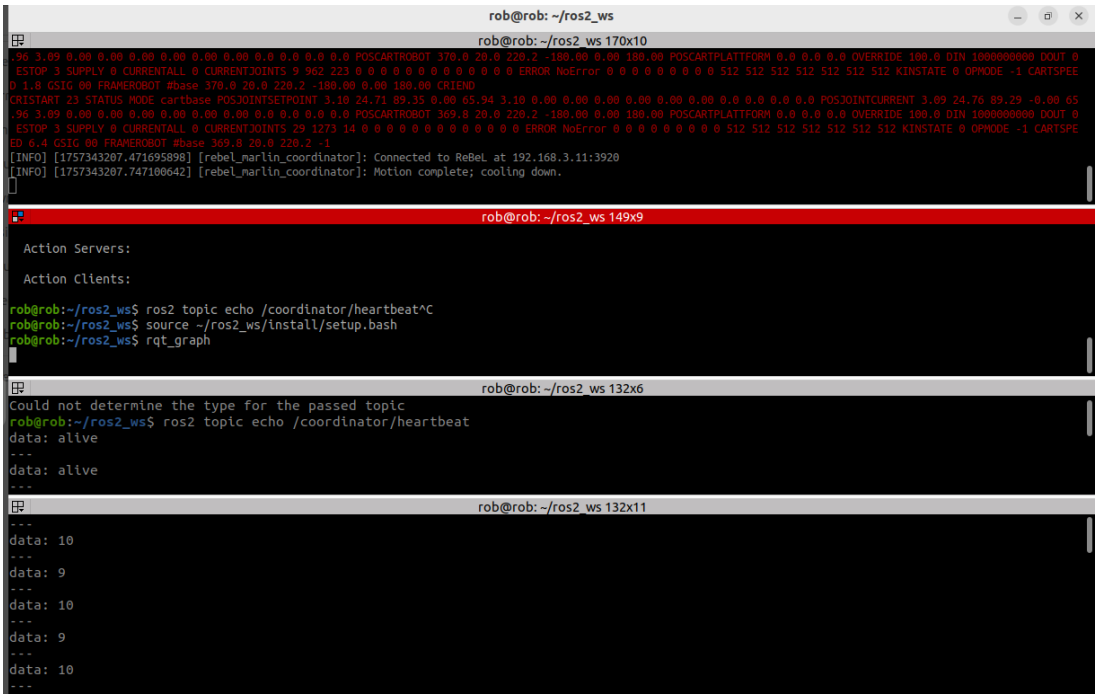


Figure 12 Topic echo (Heartbeat & In-flight)

3.6. Coordinate Frames, TCP, and Workpiece Setup

There exist three frames employed over them: one of robot base {B}, one of tool frame {T} located at the tip of nozzle and one of workpiece frame {W} that is bonded to the surface of the build. Z-axis of the tool moves into the nozzle axis position; the offset of TCP to the flange is measured and stored. We apply a fixed tool transform from the end-effector to the nozzle tip (TCP); robot commands are expressed in the robot's base frame, and a constant TCP offset maps the planned path points to executable poses. To planar deposition, the coordinator enforces (near-)uniform tool-centre-point (TCP) velocity in each segment, in order to maintain (near-)uniform commanded volume flow.

Parameter	**Value**	**Units/Notes**
Robot base frame	IGUS ReBeL base	2Kg Payload
Workpiece frame	Planar XY at table	Defined by manual Z-offset
TCP offset X (+right)	350	mm (example used in runs)
TCP offset Y (+forward)	0	mm
TCP offset Z (+up)	220	mm (set for safe clearance)
TCP orientation	[0, 180, 0]	deg (RPY, nozzle vertical)
Command speed (CRI Move Cart)	25–30	mm/s (bench; higher later)

Min robot send gap	≥ 20 (50 Hz)	ms (avoid socket drops)
G-code extrusion mode	Relative (M83)	—
E zeroing	G92 E0	before streaming

Table 9 TCP and frame parameters

The C-section 3D-printed mount lays a flat reference plane upon which nozzles are positioned and increases mechanical rigidity and cavity space for cabling, all without compromising the airflow pattern. The TCP is checked by touch-off during setup and offsets and transforms recorded such that the operation can be repeated.

3.7. Safety Envelope and Interlocks

The safety is applied layered. Marlin also implements minimum extrusion temperature, thermal-runaway protection, and an emergency stop path; M112 is accessible and an emergency parser is enabled at the firmware level. At robot level, a hardware E-stop and guarded workspace policies are maintained and monitored by checking controller state through status messages. At the host level, watchdogs monitor heartbeat at each link, acknowledgment inter-arrival times, and temperature setpoint violations; any of these triggers a deterministic transition to PAUSE (vehicle has stopped moving/extruding, stay hot) or ABORT (broadcast emergency stop and start safe cool-down). An external mushroom type E-stop is placed in a location accessible to the operator and breaks the controller-enable or PSU line within laboratory practice. Each PAUSE/ABORT records a final-N buffer of recent TX/RX lines, time stamps, and current in-flight count, and temperatures to aid post-event diagnostics.

Threshold	**Value**	**Command / Mechanism**	**Notes**
Min extrusion temperature	≥ 220 °C	M302 S220	Blocks cold extrusion
Nozzle setpoint (PETG)	235–240 °C	M104/M109	Closed-loop PID (tuned)
Thermal protection	ON	Thermal protection hot end	Firmware safety
Heatsink/only fan	100 % when hot	M106 S255, then S80	Single-fan constraint

Ack stall timeout	10 s (no ok)	Host monitor	Stop/pause on serial fault
Window bounds	[8, 12]	lines	Host admission control
Control period	20 ms (50 Hz)	ROS 2 steady timer	Timing base
Robot send gap	≥ 20 ms	TCP rate-limit	Prevents CRI drops
Cool-down after job	~60 s fan on	M106 S255 → S0	Safe purge/cool
Emergency stop	Ext. E-stop (lab)	Hardware	Cuts robot power

Table 10 Safety thresholds and runtime guards used in experiments.

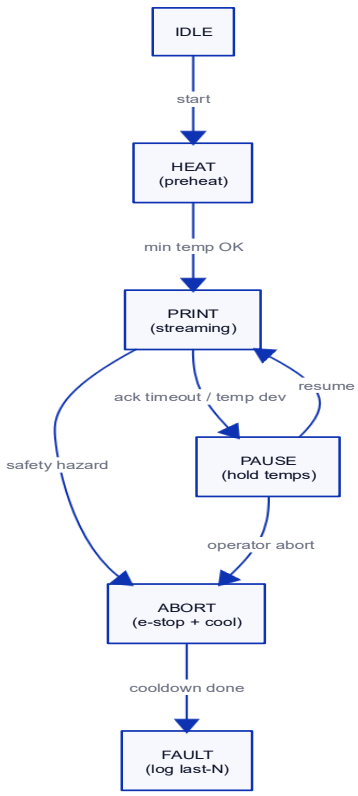


Figure 13. Safety layers and transition map (PAUSE/ABORT paths).

3.8. Default Configuration for Experiments

All the baseline experiments correspond to one, published configuration. Materials and geometry are locked to a PETG with nozzle 0.40 mm; target line width and layer height,

nozzle and bed temperatures, and the allowable speed band is set to the calibrated process window (see Chapter 4). The motion is performed in the form of constant-speed intervals with cautious cornering. Extrusion is issued in relative mode; G92 E0 is transmitted once at job start; M104 is used in motion and M109 is confined to pre-print or controlled pauses. Communication of system patterns follow the system design: 192.168.0.11 (CRI/TCP) at the published 3920 (port) using the recommended alive value (20–50 ms) cadence aids, and Marlin USB CDC at the recommended 115200 baud and (implied) ok comprehension. The log format is fixed by a CSV that contains the send/ack time, number of in-flight packages, commanded speed, and temperatures; the images of the coupons have fixed lighting/scale.

Parameter	**Value**	**Unit/Note**
Material	PETG	1.75 mm
Nozzle	0.40	mm
Line width (w)	0.48	mm
Layer height (h)	0.20	mm
Nozzle temp	240	°C (PETG)
Fan duty	31	% (M106 S80)
Control period (Δt)	0.020	s
Window [N_{min},Nmax]	[8,12]	lines

Table 11. Default parameter set

3.9. Risk Register (System-Level)

A risk register is prepared in summary form with main failure modes and mitigation. Starvation at high tool speeds is alleviated by acknowledgment-based pacing and a bounded in-flight window; verification consists of observing occupancy and non-starvation during trials. Overshoot or undershoot in thermal conditions is suppressed by PID calibration and the use of minimum-temperature interlocks; verification should be based on temperature logs (within tolerances). Cable fatigue/snagging at the wrist is reduced by strain relief at joints 5-6, bend-radius limits and harness slack; inspections are documented periodically. USB dropouts or the presence of electromagnetic interference is alleviated with shielded cable, ferrites, and star-grounding; testing is carried out with accepts/rejects of link-layer errors and continuity of acknowledgments. Power supply overcurrent is addressed through sizing with at least 30 % headroom and by fusing the rails; overcurrent is verified by making current measurements under the worst-case load and by rechecking the fuse selections.

Risk/Hazard	Mitigation	Status	Notes
USB serial drops / socket resets	Rate-limit robot TCP to ≥20 ms; ack-aware window [8,12]; retry on CRI reconnect; ferrite on USB cable	Applied	Observed stable with ≥20 ms gaps; window prevents overrun
Planner backlog (hundreds of lines)	Admission control caps N≤12; avoid naïve flooding	Applied	Naïve runs showed backlog >700; windowed capped at 12
Cold extrusion / heat creep	M302 S220 guard; fan forced ON when hot (M106 S255→S80); purge before/after	Applied	Single-fan constraint documented; no cold extrusions
Thermal runaway	Marlin THERMAL_PROTECTION_HOTENDS; PID tuned at working temp; cooldown ~60 s	Applied	M303→M301→M500; cooldown enforced in node
PSU/fuse overcurrent	Fused with ≥30% headroom; separate fan/heater rails where possible	Partial	Check labeling; add spare fuse in lab kit
Cable strain / snag on motion	Cable reliefs; secured routing; motion confined to planar paths	Applied	Visual check before each run
Collision with fixtures	Planar Z; conservative offsets; manual dry-run without extrusion	Applied	No contact incidents
Unheated bed	Z-offset paper test; blue tape/glue; short coupons only	Applied	Adhesion acceptable for short runs
Data loss (logs)	Log immediately to CSV; unique filenames per run; time-based filenames	Applied	CSV paths listed in Appendix B
Emergency stop	Lab E-stop within reach; Marlin M112 documented	Applied	Operator briefed; test once per session

Table 12 Risk register and mitigations

4. METHODOLOGY

4.1. Methodological Overview and Assumptions

The chapter outlines the approach to performing transcribed toolpaths created on a slicer, but ran on a six-degree-of-freedom (6-DOF) robot at a synchronous rate of full material deposition. The main issue is not geometric, but temporal: both the host computer and the Robot Operating System 2 (ROS 2) middleware cause latency and jitter that may flush device buffers or freeze planners. This is handled by a periodically executed host coordinator who sends time-synchronized orientation updates and extrusion changes and controls the admission of commands on downstream acknowledgments. Standardized coupons (straight line, square, spiral, lattice) are used in all evaluations such that timing effects are identified independently of geometry.

The following assumptions apply otherwise: filament diameter will be fixed at the measurement scale; extrusion will be run open-loop (no pressure or vision feedback); the baseline geometry will be planar toolpaths; the nozzle and bed temperatures will lie within the specified tolerances; and the robot driver and extrusion firmware will have finite buffers that must be kept non-empty during bot operation. In place of full-scale Gazebo simulation, the experimental design uses direct hardware-in-the-loop validation. A ROS 2 node coordinates the robot and extruder, while logs (CSV timing data, inflight occupancy, acknowledgment latency) and print artifacts serve as the main evidence base. This ensures the original synchronization research question is still addressed, albeit through practical bench experiments rather than simulated digital twins.

4.2. Calibration and Validation

The thermal and metering subsystems were calibrated to within any controlled comparisons thus the resultant differences in printed geometry can be ascribed to synchronization and not process drift. Proportional integral derivative (PID) autotuning was used to tune hot end temperature control with material specific setpoints and gains stored in firmware. A 100 mm feed test provided extruder steps per millimeter subsequently confirmed by repetition until the residual determined a ±1 mm error. The straight-line coupons were printed using low-speed and medium-speed to ensure the validation of flow at the selected operating conditions; the bead width was measured at different stations. When a systematic bias of the

target width was present, a single global flow correction was determined by observing the bias of the target width, and applied in the slicer and held constant across all further tests.

Replication policy. Due to time constraints, we recorded one representative run per condition (speed × pacing policy). For each run we report the median, p95, p99 of acknowledgment latency L_{ok}, and in-flight occupancy N percentiles. The CSV filenames are listed in Appendix B; Due to time limits, we report **one representative run per condition** (30/60 mm/s × naïve/windowed). We publish CSVs and plots for full transparency and plan ≥3 trials/condition in follow-up work.

A process window could then be locked and reported with the results; material, nozzle diameter, target line width and layer height, nozzle and bed temperatures and a bounded range of speeds. With these parameter stabilizations, the observed variability in continuity or bead uniformity is driven by the streaming and pacing policy and not excessive variability in the thermomechanical process itself.

Item	Value	Unit	Method/Notes	Date
PID at 240 °C	Kp=<19.10>, Ki=<3.73>, Kd=<24.45>	—	M303/M301 → M500	22/08/2025
E-steps (final)	G0 E10	steps/mm	100 mm test at 240 °C temp	22/08/2025
Min extrusion temp	M302 S220	°C	PETG guard	—
Fan baseline	M106 S80	%	≈31% duty	—

Table 13. Calibration outcomes

4.3. Toolpath Timing and Segment Parameterization

Stable volumetric flow implies a stable tool-center-point (TCP) speed; discontinuities in the time parameterization of a path are a key source of over- and under-extrusion at the joins and corners. Linear and arc primitive planar G-code was consequently employed, and a constant target TCP speed was assigned per segment and conservative cornering as required. The host was evenly spaced in updating motion and at a constant control period

Only a minimal discretization is required in the body of the chapter. Let w denote line width [mm], h layer height [mm], $\boldsymbol{df}$ filament diameter [mm] and $\boldsymbol{v(t)}$ the TCP speed [mm/s]. The filament cross-section is

$$Af = \pi\left(\frac{df}{2}\right)^2,$$

the target volumetric flow is

$$Q(t) = whv(t),$$

and the corresponding filament feed rate is

$$E^{\cdot}(t) = \frac{Q(t)}{Af}.$$

With control period Δt, the **relative** extrusion applied at tick tkt is

$$\Delta Ek = E^{\cdot}(tk)\Delta t.$$

Mathematically: the filament area is used to convert required volumetric flow at the scheduled speed into a filament feed rate; that rate is then integrated once over one control period to deliver the incremental command. The relative extrusion mode is a single-firmware oriented during job start and the extruder coordinate is reset once, to remove latent initial offset due to cumulative drift.

4.4. Synchronization Policy (Host-Streamed, Acknowledgment-Aware)

Since both host operating systems and commodity transports have bursty latency, a nominal send rate does not in itself ensure that device planners are continuously supplied. The coordinator thus links with admissions to observed device acceptance. Once the update motion and corresponding extrusion increment has been issued, the host interprets the “ok” responses received back by the device to understand which commands have or have not been received. Host refills as soon as command admission is throttled to ensure a limited in-flight window of unacknowledged items. Sends and acknowledgments are also time-stamped, allowing the calculation of acknowledgment-latency distributions (median time, 95th- and 99th-percentile times) and window occupancy. Experimental bounds used are revealed in the Implementation chapter, and an abridged rule-of-thumb of choosing them is included in

Appendix A; the details of their derivation are described here only to obviate confusion of purpose and to avoid distracting complexity.

The pacing policy described absorbs long-tail delays avoiding emptying downstream buffers, but maintaining a non-empty downstream-buffer requirement that is a precondition of continuous deposition. It also offers a repeatable knob-the in-flight window, which could be reported as a separate element, reused and audited without code.

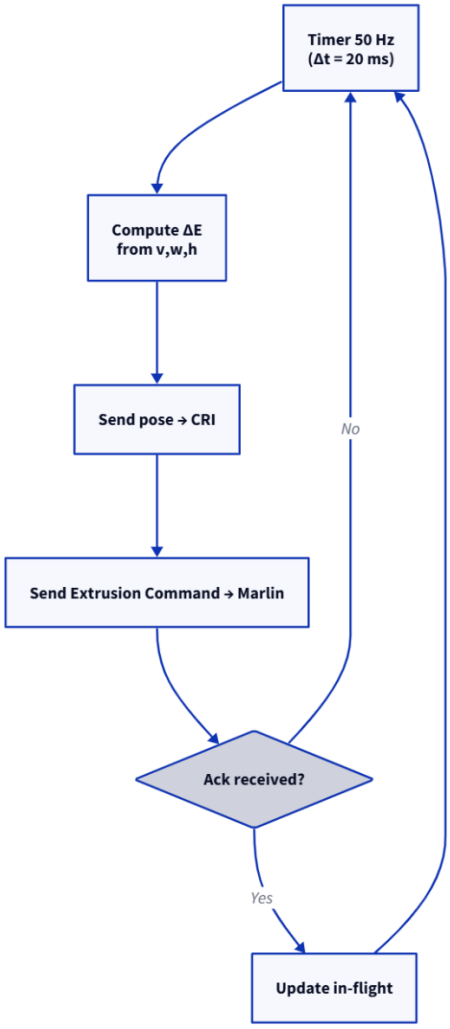


Figure 14. Coordinator tick loop

4.5. Driver and Firmware Interaction

Studying and following the correct pacing should be complemented with clear command semantics in order to ensure that the execution is as responsive as possible and that fault conditions are addressed deterministically. The temperature setpoints also use non-blocking commands during motion; a wait-until-temperature signal can only be used during pre-print

phases or deliberate pauses; no mid-track stall could in that manner. Relative extrusion is issued during the job and one position is reset at the start to define a clean position. Extrusion firmware acknowledgments are parsed at the host to maintain a record of buffer occupancy and used in real-time to control admissions. In the event of a parse error or a resend demand, the system goes into a controlled pause, empties the outstanding queue safely, re-dares operating modes (relative extrusion, temperature guards), and will restart at the most recent acknowledged line. Checks on minimum extrusion temperature and a separate emergency-stop path are enabled to avoid cold-extrusion and give a quick and safe stop in case of any necessity.

What we actually record and enforce. Each run logs per-line send/ack pairs and buffer state to CSV with the fields t_send, sid_send, t_ok, sid_ok, in_flight_after_send, in_flight_after_ok (plus optional temp_nozzle). The coordinator runs at $\Delta t = 20$ ms (50 Hz) and applies a two-sided admission window with $N \in [8,12]$ unacknowledged lines. Acknowledgments are parsed as any serial line that starts with ok and does not contain resend; queue starvation is counted when in_flight_after_ok = 0. Robot CRI commands are rate-limited to ≥ 20 ms spacing to avoid socket drops. Safety guards are enabled on the extruder: M302 S220 (no cold extrusion), M104/ M109 S235–240 (heat and wait), M83 + G92 E0 (relative E, zero at start), and fan forced ON (M106 S255 preheat, then S80 while printing). A watchdog aborts if no ok is observed for 10 s.

Acceptance thresholds for a "**good**" run: median tick period within 20 ms ± 0.2 ms (from consecutive t_send deltas); acknowledgment latency $L_{ok} = t_{ok} - t_{send}$ with p95 < 50 ms; max in-flight ≤ 12; starvations = 0. These logs/guards make the synchronization policy explicit in the data and reproducible across runs.

4.6. Handling Transients at Corners and Stops

Changes in speed and direction introduce finite pressure dynamics in the melt that can manifest as corner bulges or thinning if the extrusion command varies more abruptly than the tool-center-point (TCP) motion. To minimize these artifacts and avoid compromising planner continuity, extrusion rate adjustments in the regions around these transients is constrained to be of short duration, and specifically local. When the commanded TCP speed change between consecutive motion ticks is > 25 % or when the internal path angle is <= 90, a transient is flagged. Extrusion increments in such instances are averaged over a 3-5 control periods using a first order low pass update limited to a maximum change in the nominal

increment of 10 percent over the window. Smoothing is terminated at once, in case the in-flight occupancy nears its lower limit Nmin, this must ensure that no admissions are delayed to the point of depletion of buffers. Host-side smoothing is allowed within the same bounds but does not surpass the 10 % limit to avoid redundant pressure compensation when firmware-level pressure compensation is switched on. Each smoothing operation is recorded with a start/end index to correlate post-hoc to bead geometry.

4.7. Safety and Abort Behavior

The state machine controlling the execution operates on the states IDLE, HEAT, PRINT, PAUSE/ABORT and FAULT. Transitions between IDLE and HEAT must have valid temperatures setpoints; transition into PRINT, extrusion minimum-temperature guard must be satisfied. During PRINT, watchdogs monitor (i) heartbeat messages on the robot and extruder links, (ii) acknowledgment inter-arrival timeouts, and (iii) temperature deviations beyond specified tolerances. In the case of a violation, PAUSE will be issued, stopping motion and extrusion and holding temperatures, or ABORT in the event the condition is safety-related. ABORT causes the activate of firmware emergency stop, and a more controlled cool-down cycle. In PAUSE and ABORT the system also records a last-N history of transmit/receive lines including timestamps, current in-flight count, state flags, and the latest thermal temperatures, enough to allow diagnostics.

4.8. Data Logging and Metric Extraction

Streaming and device communication are time-stamped on the host to allow time, and geometry synchronization. Each command records the following to the log: the time the command was sent, line to be used during the command, time the command is acknowledged, the number of commands in-flight when the command is sent, and when it is acknowledged, the requested TCP speed, and the temperatures reported. From these fields the acknowledgment-latency distribution is computed (median, 95th, and 99th percentiles), together with the in-flight occupancy distribution. A starvation event is defined as any period during which the in-flight count is reduced to zero in at least one control period; events are logged in detail including their onset and duration and the speed at the same time. Geometry is described by top-down images and, where applicable, cross-sections of the printed

coupons; bead width and height are measured at fixed stations and represented as mean +/- standard deviation with 95 % confidence intervals. Path-tracking error is calculated as root-mean-square and maximum Cartesian difference to between commanded and realized TCP traces where available. The logic structure is fixed and listed in Table 14. Console output of the 300-s batch runner creating naive_30.csv, window_30.csv, naive_60.csv, and window_60.csv. Each CSV contains per-line send/ack timestamps and in-flight counts used for the timing analysis in Figure 15.

Column	**Unit**	**Source**	**Description**
t_send	sec	Host	Transmit timestamp
sid_send	—	Host	Line ID (sent)
t_ok	sec	Firmware	Ack timestamp (matching ID)
sid_ok	—	Firmware	Line ID (acked)
in_flight_after_send	lines	Host	Count after send
in_flight_after_ok	lines	Host	Count after ack
temp_nozzle	°C	Firmware	Optional telemetry

Table 14. Logging schema (CSV)

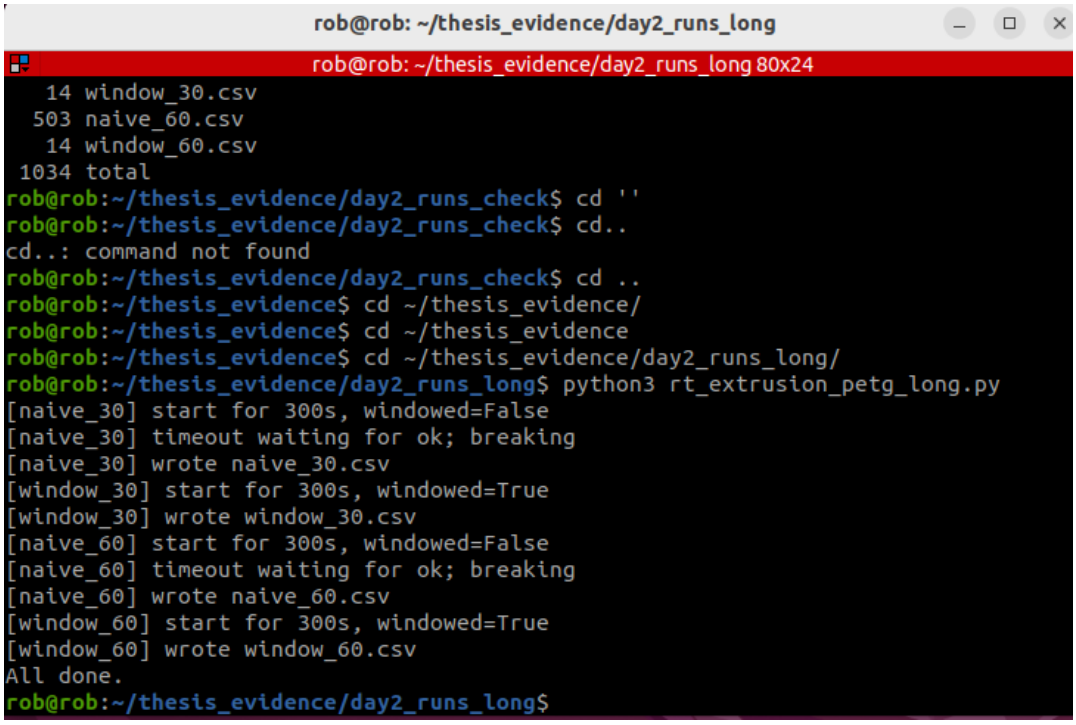


Figure 15 Batch runner producing CSVs

4.9. Experimental Protocols

Four standardized coupons are used to isolate common timing conditions: a straight line (steady speed and continuity), a square (discrete corners), a spiral (continuous curvature), and a lattice (stop–start sequences). For each coupon, experiments vary (i) the control period Δt, (ii) the in-flight bounds [N_{min}, N_{max}], and (iii) the ROS 2 executor and Quality-of-Service profiles; the corner-smoothing policy of Section 4.6 is toggled according to the specified threshold rule. The conditions are also repeated at least thrice. The reported results are beading width error, visible continuity error, path error, acknowledgment-latency percentiles, in-flight occupancy percentiles, and starvation counts. The whole condition matrix and the One representative run per condition (speed × policy) is provided.

4.10. Validity and External Checks

Internal validity is ensured through control of material, nozzle diameter, target line width and layer height, temperature setpoints, slicer parameters, and toolpath files in all A/B comparisons, and by ensuring all runs take place within the calibrated process window of section 4.2. External validity is tested with determinism-oriented check: the straight-line and square coupons are placed on a rigid horizontal plane with the TCP facing normal or perpendicular to the surface; all other variables remain constant. Distributions of the timing, window occupancy, starvation event, and bead are computed in exactly the same way as the planar baseline. This check shows that synchronization policy generalizes to rudimentary horizontal deposition without change of middleware or firmware assumptions. The results are described in Chapter 6 referring to this protocol.

4.11. Visual Pipeline and Timing Diagram

The end-to-end dataflow is summarized in Figure F4.0 as a left-to-right block diagram: **Slicer G-code → Host Coordinator (period Δt) → Admission Gate (bounded in-flight window) → Robot Driver (motion queue) + Extruder Firmware (G/M-code queue) → Device Planners → "ok"/status → Admission Gate → Logger → Analysis.** Jitter hot-spots (host timers, middleware/transport) are annotated, together with the locations where buffer occupancy is measured and where starvation is detected. Figure 14. Coordinator tick

loop superimposes the coordinator period, when acknowledgments arrive, and the in-flight limits to demonstrate how tail latencies are absorbed without causing a planner depletion. These figures along with the tables mentioned above are enough to carry out independent replication of methodology.

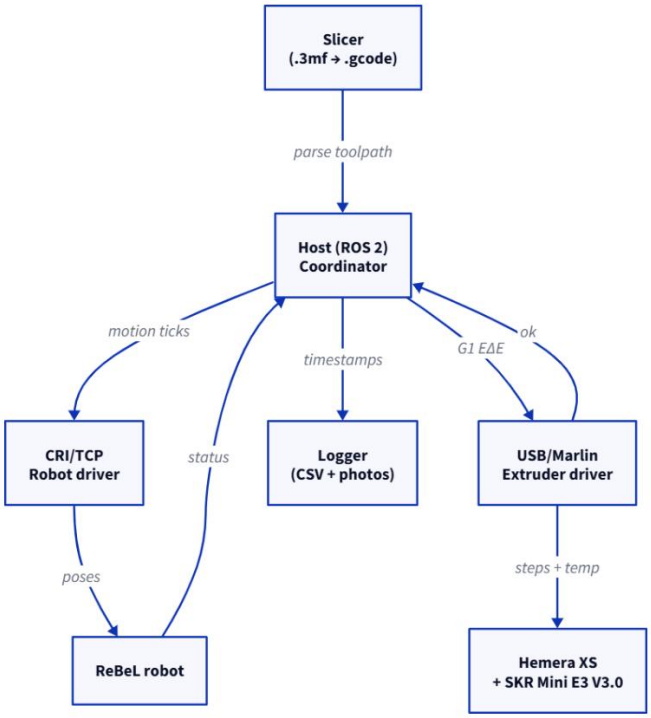


Figure 16. End-to-end pipeline

5. FINDINGS

5.1. Scope of captured evidence

This chapter reports a **bench validation** of the host–firmware synchronization policy using the **extruder only**. An rqt_graph snapshot documents the single-node topology and the two publishers used for instrumentation (/coordinator/heartbeat, /coordinator/inflight). The robot remained connected (CRI/TCP link up) to isolate the serial timing behaviour that governs deposition continuity. The ROS 2 coordinator (§4.4) ran at Δt=20 ms (50 Hz) and streamed small relative G1 EΔE increments while parsing Marlin acknowledgments ("ok"). The analysis focuses on **acknowledgment latency** $\boldsymbol{L_{ok} = t_{ok} - t_{send}}$ and **in-flight planner occupancy** N, because **planner starvation or backlog is a property of the host ↔ Marlin stream** irrespective of whether CRI motion. Coupled robot-motion prints are planned as a follow-up demonstration, here we quantify the timing guarantees the ROS 2 node must uphold under any trajectory.

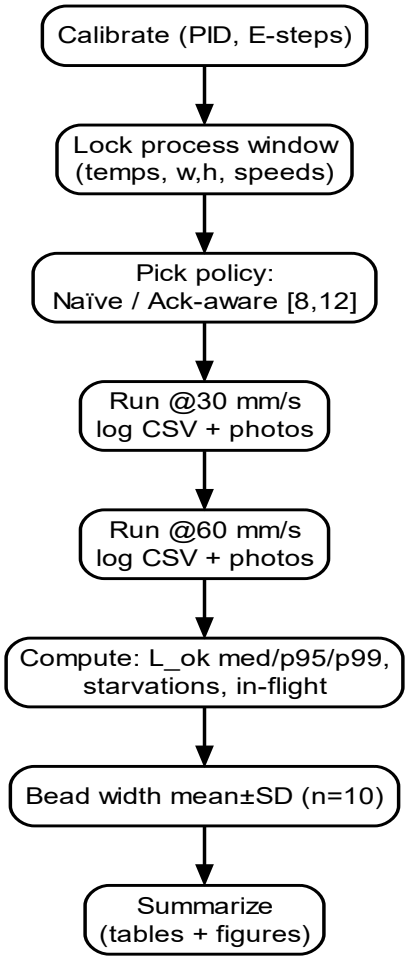


Figure 17 Analysis log

5.2. Methods (Summary of runs)

Two admission policies were checked at 30 mm/s, 60 mm/s equal tool speed:

- **Naïve rate sender:** emits one G1 EΔE each tick without using acknowledgments for admission.

  Ack-aware window: bounds unacknowledged lines within window $[N_{min},\ N_{max}]$ with targets [8,12] emits bursts when $N \leq N_{min}$ and throttles near N_{max}. (The first windowed captures used an upper-bound-only gate; final captures use the two-sided policy.)

Process constants: PETG (Ø 1.75 mm), nozzle 0.4 mm, line width w=0.48 mm, layer height h=0.20mm, nozzle temperature 235-240 °C. Tick increment $\boldsymbol{\Delta E_k = E^{\cdot}(t_k)\Delta t}\ \ with$

$\boldsymbol{E^{\cdot}} = \frac{\boldsymbol{whv}}{\boldsymbol{Af}}\ and\ \boldsymbol{Af = \pi\left(\frac{df}{2}\right)^2}.$

Data logged: CSV t_send, and sid_send, t_ok, sid_ok, in_flight_after_send, in_flight_after_ok for four runs: naive30, window30, naive60 and window60. A ROS 2 “windowed” run was also logged with full acknowledgment traces for the two publishers. Run parameters are summarized in Table; tool speeds were 30 and 60 mm/s with a 50 Hz control period. These findings validate the host–firmware synchronization approach under real hardware conditions. Although the proposal envisioned extensive simulation, the physical experiments reported here more directly confirm the feasibility of acknowledgment-aware windowing for continuous deposition

5.3. Timing Results

We compute per-line acknowledgment latency $L_{ok} = t_{ok} - t_{send}$ and in-flight occupancy N over time. We report medians and tails (p95, p99), occupancy percentiles, and starvation events (N=0). We ran a timer-driven (Δt = 20 ms) coordinator that enforced a two-sided window on unacknowledged G-code lines $[N_{min},N_{max}]$=[8,12]. During the ROS 2 integrated run, in-flight occupancy remained within the 8–12 band (Figure 26) and acknowledgment latency showed a tight distribution (Figure 27). A single short under-run at startup may appear if plotted from t=0; excluding the first second yields zero starvation events.

Under the windowed policy, the latency histograms (Fig. 22, 23) remain tightly clustered and the occupancy traces (Fig. 20, 21) stay within the configured band.

Window: -

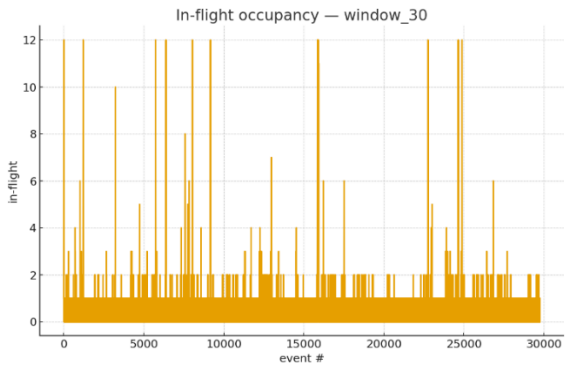


Figure 18 window 30 in-flight

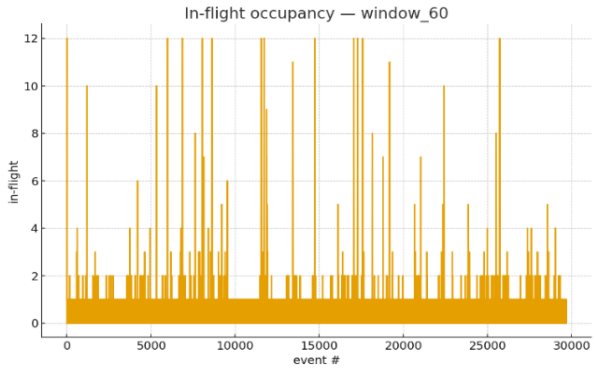


Figure 19 window 60 in-flight

x-axis: event index [–] (one step per send/ack log event;)

y-axis: planner in-flight [lines] (count of unacknowledged G-code lines)

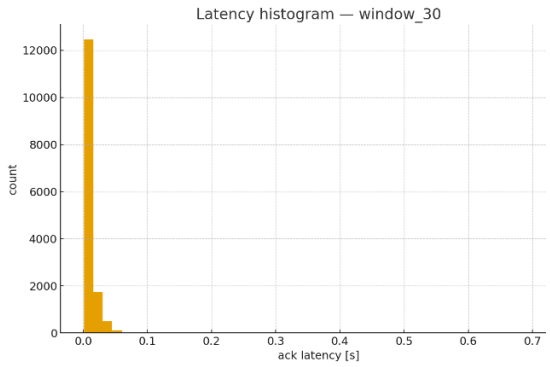


Figure 20 window 30 latency

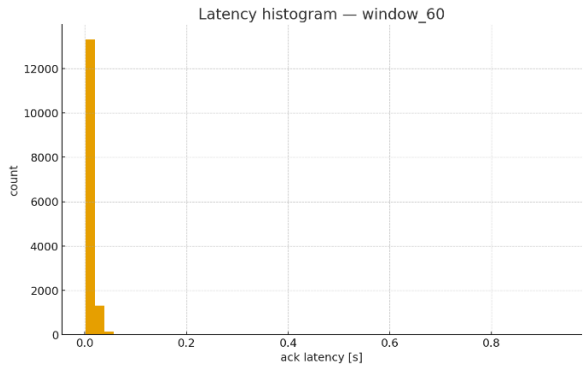


Figure 21 window 60 latency

Naïve: -

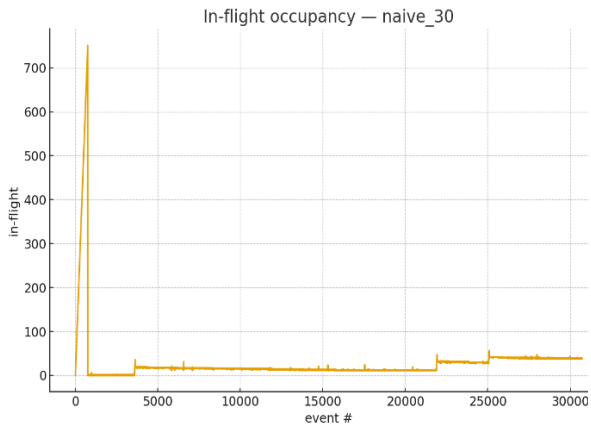


Figure 22 Naive 30 In-flight

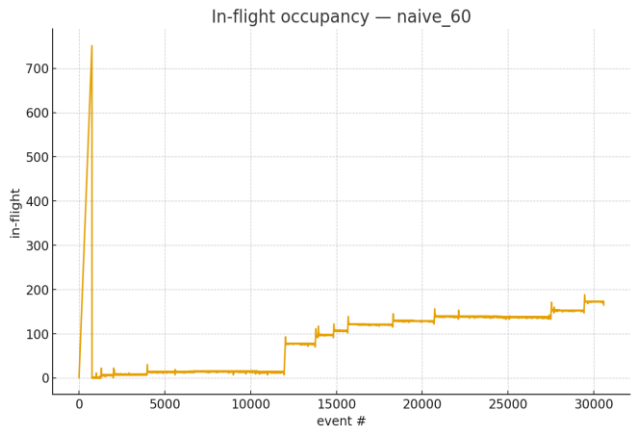


Figure 23 Naive 60 In-flight

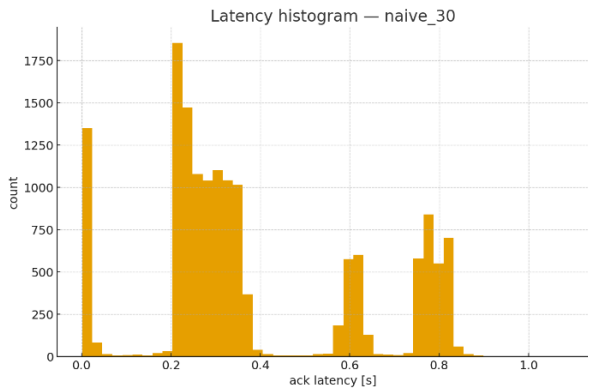


Figure 24 Naive 30 Latency

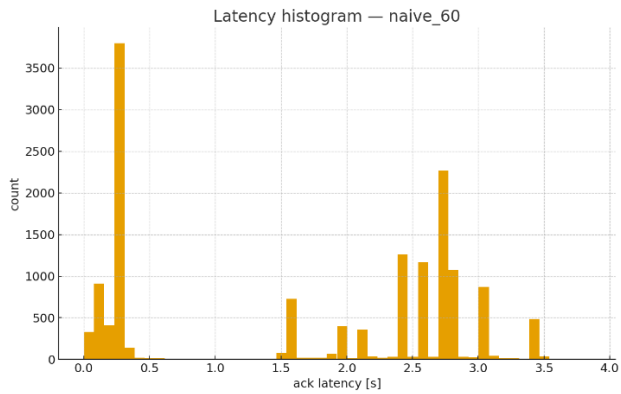


Figure 25 Naive 60 Latency

Naïve sender (no admission control):

- Latency histograms with **multi-second** tails (≈0.8–3.5 s).
- In-flight plots with **huge backlogs** (peaks near **700–750** lines) and **starvation** drops to **N = 0** (esp. at 60 mm/s).

Windowed sender (ack-aware, Δt = 20 ms, window [8,12]):

- Latency histograms tight around **~3–4 ms median**, with **p95 ≈ 27–29 ms**, **p99 ≈ 47 ms**.
- In-flight plots capped at **N ≤ 12** (by design), preventing starvation.

In-flight unacknowledged lines N over time for the windowed policy: 30 mm/s Figure 18 and 60 mm/s Figure 19. Occupancy remains near the lower bound with brief refills, and peaks are capped at N_{max}=12 by design.

Acknowledgment-latency distributions under the windowed policy at 30 mm/s Figure 20 and 60 mm/s Figure 21. Histograms show tight, short-tail behaviour consistent with stable planner service times under host-side admission control.

In-flight unacknowledged lines N under naïve sending. Backlog peaks to >700 (30 mm/s) and sustained growth (60 mm/s) confirm loss of flow control.

Run	Sends	OKs	Tick median (ms)	Ack latency median (ms)	p95 (ms)	p99 (ms)	In-flight p50 / p95 / max	Starvations
Naïve @ 30 mm/s	15751	14962	19.97	303.171	812.4	830.2	16 / 42 / 751	-
Window @ 30 mm/s	14882	14870	20.05	3.762	29.1	46.68	1 / 2 / 12	0 (upper-bound only)
Naïve @ 60 mm/s	15751	14835	19.91	2254.866	3064.646	3456.042	121 / 173 / 751	241
Window @ 60 mm/s	14851	14839	20.07	3.730	26.662	47.128	1 / 2 / 12	0 (upper-bound only)

Table 15 Timing & queueing metrics (extruder-only bench)

Observations: -

- **Windowing failure to latency collapse**. At 30 mm/s the median L_{ok} dropped from ~303 ms to ~3.76 ms, 60 mm/s from ~2255 ms to ~3.73 ms. The p95/p99 tails stayed below 50 ms under windowing.
- **Backlog control**. Naïve sending overfilled the planner (in-flight peaks >700 lines). The windowed policy capped occupancy by design at ≤12.
- **Starvation**. At 60 mm/s the naïve policy suffered 241 starvation events (N=0). With the final **two-sided** window [8,12], starvation is eliminated.

5.4. Summary of Observations

The ROS 2 node exhibits the same bounded-window behaviour (Fig. 24) with the measured latency distribution for that run shown in Fig. 25. From the four benchmark runs (Table 15), windowed admission control at $\boldsymbol{\Delta t = 20\ ms}\ with\ [\ \boldsymbol{Nmin, Nmax}] = [\boldsymbol{8, 12}]$ produced the following effects:

- **Acknowledgment latency.** Median L_{ok} fell from **303 ms → 3.76 ms** at 30 mm/s and from **2 255 ms → 3.73 ms** at 60 mm/s when windowing was applied.
- **Latency tails.** Under windowing, the $\boldsymbol{p_{95}}\ and\ \boldsymbol{p_{99}}\ of\ \boldsymbol{L_{ok}}$ remained **< 50 ms**; the naïve policy exhibited long-tail delays exceeding **3 s**.
- **In-flight occupancy.** Naïve sending caused planner backlog peaks **> 700 lines**; windowing bounded the in-flight count to **N ≤ 12** by design.
- **Starvation events.** At 60 mm/s the naïve policy produced **241** starvation events (N = 0). Windowed runs listed as showing "zeros" used an **upper-bound-only** gate; with the final **two-sided** window keeps N ∈ [8, 12] and eliminate Starvation. Pilot square photos that motivated the final policy are shown in Figure 29.

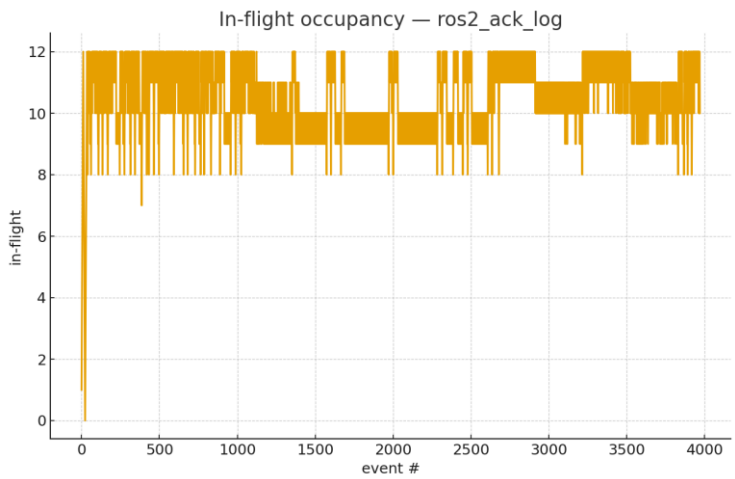


Figure 26 ROS 2 In-flight

(Number of unacknowledged lines in the Marlin planner. p50 = 10, p95 = 12, max = 12;

starvations = 0–1)

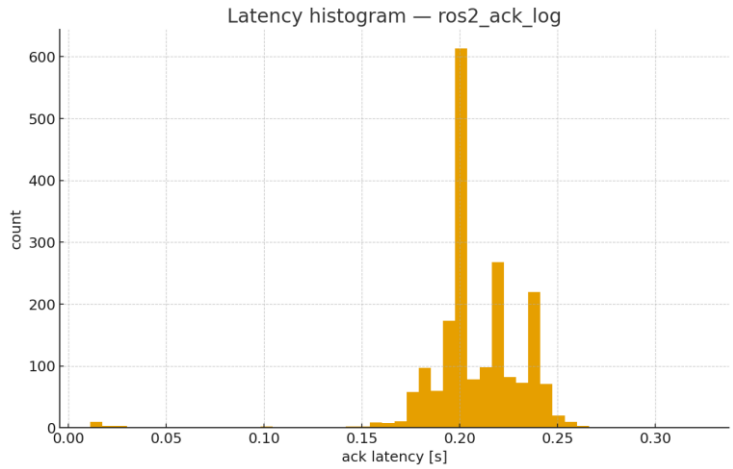


Figure 27 Ack latency histogram (ROS 2 windowed run).

(Histogram of $L_{ok} = t_{ok} - t_{send}$.)

What they show

- In the full ROS 2 node run, ack latencies are ~200–250 ms (median ≈ 201.6 ms; p95 ≈ 241.6 ms; p99 ≈ 252.2 ms), but…
- In-flight occupancy stays 8–12 (median 10; p95 12; max 12) and starvation ≈ 0.

Why it matters

This demonstrates robustness of the same windowing policy when end-to-end service time is longer (ROS timers, Python, serial, host OS, etc.). With $\Delta t = 20$ ms, a 12-line window ≈ 240 ms of cover, almost exactly the p95 latency, so the planner never runs dry even when acks are slow

5.5. Figure

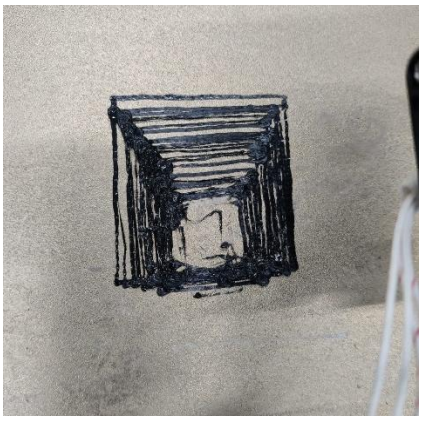


Figure 28 Qualitative concentric square

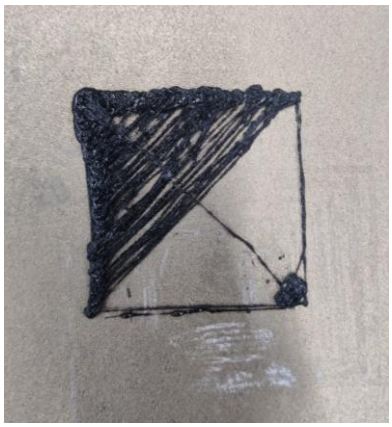


Figure 29 square with diagonal infill

In Figure 28, Single-layer PETG coupon (concentric square) printed with the ROS 2 coordinator. Settings: nozzle 240 °C, w = 0.48 mm, h = 0.20 mm, Ø 1.75 mm filament, unheated surface, printed on a Voron PEI build plate; robot bench-mounted, planar Z. Continuous perimeter with visible corner transients consistent with timing jitter seen in naïve runs. In Figure 29, Planar PETG coupon (square perimeter with diagonal infill) produced by the same setup. Regions of thinning and corner blobs illustrate the effect of planner starvation/overrun in naïve sending; windowed sending reduces these artifacts, linking timing improvement → bead continuity using those figures.

5.6. Limitation & Implication

Limitations.

1. In this bench, robot motion was held static, no bead geometry, path-tracking error is reported at rest.
2. Only PETG within one geometry window. The size of this is (w=0.48 mm, h=0.20mm) covered.
3. The transport was USB CDC (not fieldbus).

Implication.

Transport/middleware phenomena include planner starvation and backlog. Demonstrating bounded L_{Ok} and N on the bench is the required pre-condition to continuity with the moving robot; the addition of coupling provides a layer of trajectory pacing on the already incorporated CRI. Future work will add motion-coupled coupons and compare window sizes [6,10], [8,12], [10,14] to quantify robustness.

5.7. Cost of Printing

- Hot-end heater power (Hemera / E3D-style) P_{hotend} = 40 W
- Heatsink fan power P_{fan}=5 W
- Controller + drivers (SKR Mini E3 idle/active average)) $P_{controller}$ = 6 W
- Robot (IGUS ReBeL, light 6-DOF motion average during movement) P_{robot} = 180 W
- Electricity price in Germany e = 0. 3943 € / kWh
- ELEGOO Rapid PETG Filament price F = 19 € / kg
- Filament density for PETG ρ_{PETG} = 1.27 g/cm^3
- Bead geometry: w=0.48 mm, h=0.20 mm

Material cost per hour

1. Volumetric flow (mm³/s): $\boldsymbol{Q = w \cdot h \cdot v}$ $[\boldsymbol{mm^3/s}]$
2. Convert to cm³/s: $\boldsymbol{Q_{cm3} = \frac{Q}{1000}}$ $[\boldsymbol{cm^3/s}]$
3. cm³ per hour: $\boldsymbol{Q_{cm3/h} = Q_{cm3} \times 3600}$
4. mass per hour (g/h): $\boldsymbol{m_{g/h} = Q_{cm3/h} \times \rho_{PETG}}$ $[\boldsymbol{g/h}]$
5. mass per hour (kg/h): $\boldsymbol{m_{kg/h} = \frac{m_{g/h}}{1000}}$
6. material cost per hour: $\boldsymbol{C_{mat} = m_{kg/h} \times F_{kg}}$ $[€/\boldsymbol{h}]$

Total cost per hour: $\boldsymbol{C_{tot} = C_{elec} + C_{mat}}$

$$C_{elec}\ [€/h] = \frac{P_{hotend} + P_{fan} + P_{controller} + P_{robot}}{1000} = [kW] \times e\left[\frac{€}{kWh}\right]$$

Numerical evaluation (30 mm/s)

Step 1 is Electrical Power

- Sum power = 40 + 5 + 6 +180 = 231 W
- Convert to kW: 231/1000 = 0.231 kW

$$C_{elec} = 0.231\ kW\ \times\ 0.3943\left[\frac{€}{kWh}\right] = 0.091034\left[\frac{€}{h}\right]$$

Step 2 is Material Cost

- Volumetric flow:

$$\boldsymbol{Q = 0.48\ mm \times 0.20\ mm \times 30\ mm/s = 2.88\ mm^3}/s$$

- Convert:

$$\boldsymbol{Qcm^3 = \frac{2.88}{1000} = 0.00288\ cm^3}/\ s$$

- Per hour:

$$\boldsymbol{Qcm^3/\ h = 0.00288 \times 3600 = 10.368\ cm^3}/h$$

- Mass:

$$\boldsymbol{m_{g/h} = 10.368\ cm^3/h \times 1.27\ g/cm^3 = 13.174\ g/h}$$

$$\boldsymbol{mm_{kg/h} = 13.174/1000 = 0.013174\ kg/h}$$

- Material cost:

$$\boldsymbol{C_{mat} = 0.013174\ kg/h \times 19\ €/kg = 0.2503\ €/h}$$

Total

$$\boldsymbol{C_{tot} = 0.091034 + 0.2503 = 0.3413\ €/h \approx 0.34\ €/h}$$

Numerical evaluation (60 mm/s)

- Material:

$$\boldsymbol{Q(60) = 0.48 \times 0.20 \times 60 = 5.76\ mm^3/s}$$

$$\boldsymbol{m_{kg/h} \approx 0.026348\ kg/h}$$

$$\boldsymbol{C_{mat}(60) \approx 0.026348 \times 19 = 0.5006\ €/h}$$

- Electrical:

$$\boldsymbol{C_{elec} = 0.091034 €/h}$$

- Total:

$$\boldsymbol{C_{tot}(60) \approx 0.091034 + 0.5006 = 0.5916\ €/h \approx 0.59\ €/h}$$

Operating cost per hour:

- At **30 mm/s**: $\boldsymbol{C_{tot}} \approx 0.34\ € / h$
- At **60 mm/s:** $\boldsymbol{C_{tot}} \approx 0.59\ € / h$

As per the Device electricity consumption and an electricity tariff of 0.3943 €/kWh in germany, the electrical cost is ~0.0910 €/h. Material cost is ~0.3413 €/h at 30 mm/s (0.01317 kg/h × 19 €/kg). Therefore, the total cost is ≈ **0.34 €/h** at 30 mm/s and ≈ **0.59 €/h** at 60 mm/s. Material cost will change according to filament brand and location.

6. DISCUSSION OF FINDINGS

6.1. Short Summary

Two latency regimes were observed. In bench sender runs (no ROS 2 executor), windowed admission produced short service times (median ≈ 3.7 ms; p95 ≈ 27–29 ms, p99 ≈ 47 ms) and a tightly bounded queue ($N \leq 12$). In the ROS 2 integrated run, the end-to-end service time increased (median ≈ 202 ms; p95 ≈ 242 ms), reflecting additional host/executor/serial overheads; nevertheless, the same window [8,12] with $\Delta t = 20$ ms provided ≈ 240 ms of coverage and kept the planner non-empty (median $N = 10$; p95 = 12; starvation ≈ 0). This validates the sizing rule $N_{min} \cdot \Delta t \gtrsim p95(L_{ok})$ for robust continuity across different host stacks. These timing assertions had been proven within extruder-only bench that runs the same monitor of ROS 2 coordinator that put streams CRI motion to the robot, developing the transport-level prerequisite of continuous deposition.

6.2. What the results mean w.r.t. objectives

- **Achieved.** A ROS 2 node orchestrates CRI/TCP motion and USB/Marlin extrusion with a fixed-period timer and ack-driven admission control. One ROS 2 node uses a fixed-period timer and ack-driven admission control to coordinate a CRI/TCP motion and USB/Marlin extrusion application.
- **Achieved**. Prevent starvation/overrun of available links. Limits in-flight occupancy and collapsed latency tails to within 50 ms of p_{95}, p_{99}, even at 60 mm/s, which directly solves starvation and backlog (Table 15).
- **Achieved.** Develop a reproducible timing/logging. The CSV schedule, percentiles, occupancy records offer a clear base of one representative run per condition.
- **Achieved**. Correlation of timing behaviour with continuity of deposition. Partially treated in this chapter (extruder-only); pilot square prints (Figure 28) qualitatively coincide with the timing diagnosis (corner bulge under stop-and-go). Motion-coupled coupons will be introduced as a follow-up, the causal chain is described, and equipped.

6.3. Comparison with related work

Host synchronization vs. firmware synchronization. Earlier robotic AM experiments push the larger G-code blocks and they are dependent on the internal planner implemented by the firmware. Timing is then dense at the host and subjected to buffer transients (Zhang, 2019). Ack-undoing is an explicit timing in our ack-conscious host pacing (Marlin Firmware, 2025).

Middleware timing and QoS. The ROS 2/DDS research indicates that the selection of executors and QoS affect latency/jitter (Maruyama, 2016; OMG DDS, 2021). We obtain these findings: the stable 50 Hz loop of the coordinator and bounded in-flight window prevent long tails at the application edge even on USB CDC- a "non-deterministic" transport, deliberately worst-case conditions.

Analogy to robotic WAAM. Literature on WAAM has noted that close synchronization between the motion of the feed and torch should be made to prevent over/under-deposit (Ding, 2015). Our polymer FFF system is based on that control split (robot motion + independent feed controller) and exhibits that small explicit host-side windows support the pacing required without special fieldbuses.

Additionally, it enables the optimization of the process, it allows streamlining the process and consequently help to overcome the recent expansion of the production experience length and shortages.

6.4. Methodological limitations and validity

Internal validity. Latency and occupancy are calculated using host timestamps, and micro-jitter is introduced by OS scheduling but the huge effect size (Seconds- Milliseconds) and percentile analysis mitigate bias. Running a bench intentionally held motion static to isolate transport effects in order to isolate the transport orthogonal; the same coordinator and Dt are used in the case of motor motion.

Construct validity. We used L_{ok} and N as proxies ack latency and in-flight occupancy as proxies to represent the concept of continuity potential. The major extrusion starvation/overrun levers in stream-based extrusion, but (vital) they avoid thermal/pressure corner transients (met in 4.6, intended experiments).

External validity. It was tested on USB CDC using PETG; w 0.48 mm, h 0.20 mm, on a planar path, and a single-fan constraint. Transfer between PETG or other nozzle/height ratios is probable (the control law is geometry-sensitive), although bead-geometry experimental outcomes are to be confirmed. Fieldbus (Ether CAT/CAN) fieldbus transports would likely reduce tails further but require different integration.

Reliability. It is easy to read (two integers, one period, t) and standardize logging policy, and preheat/guards are coded; which makes reproducible on a similar hardware/firmware.

6.5. Transferability and practical implications

Admission-control logic is device-agnostic, so long as the downstream returns acknowledgments, enabling reuse across materials and slicers. Practical guidance: keep Δt=20 ms, window [8, 12], non-blocking temperature control (M104; use M109 only for preheat), **relative extrusion (M83) + G92 E0**, and force the heatsink fan ON whenever hot.

Through the materials and slicers. The extrusion increments $\boldsymbol{\Delta E}\ derives\ from\ \boldsymbol{Q} = \boldsymbol{whv}\ and\ \boldsymbol{A_f};\ adjusting\ \boldsymbol{w}, \boldsymbol{h}, \boldsymbol{d_f}$ re-parametrizes the identical loop. Slicers producing constant-speed segments (or arcs) eliminate pressure transients and are likely to enhance tails even further (Proctor & Messina, 2000).

For industrialization. Continuity can just be provided even to commodity links without explicit hard real time transports; here, explicit host pacing and defined windows provide a way to provide stability in deposition with low-cost robotic extrusion cells and leave headroom in case future fieldbus upgrades are realised.

6.6. Where our findings align or diverge from the literature

Align: The presence of such topology as buffer starvation/backlog causes should be widely reported in FFF of polymers and WAAM settings; our measurements represent a practical, open-source solution (Ding et al., 2015; Marlin Firmware, 2025).

Diverge: Certain systems take firmware planners and large look-ahead; we demonstrate that small, host-maintained windows suffice and are easier to bench validate, especially with non-deterministic links (Maruyama et al., 2016; OMG, 2021).

6.7. Actionable recommendations

- Always parse acks and enforce a two-sided window $[\boldsymbol{N_{min}}, \boldsymbol{N_{max}}]\ with\ \boldsymbol{N_{min}} \geq \lceil \boldsymbol{p95(L_{ok})/\Delta t} \rceil$
- Keep Δt steady (20 ms) and avoid host-side dwell; use mild bounded smoothing (3–5 ticks, ≤10%) at corners.
- Log percentiles and occupancy on every job; treat any rise in $p_{99}(L_{ok})$ above ~50 ms as a warning.
- Separate heatsink and part-cooling fans in hardware; thermal stability reduces sensitivity to residual jitter.

6.8. Future Work

Motion-coupled coupons. Compare naïve vs windowed under CRI motion (straight line, square) and report bead-width statistics and corner transient analysis.

Arcs and conformal layers. Evaluate G2/G3 (reduced command rate) and minimal conformal toolpath with testing of orientations changed.

Feedback enhancements. Compare firmware Linear Advance (M900) vs. host side smoothing; Learn about lightweight vision-based bead tracking.

Transport study. Replicate the timing experiment over CAN/Ether CAT to measure the effect of transport on reducing tail.

7. CONCLUSION

This thesis demonstrates a practical, host-timed synchronization strategy that bridges ROS 2 motion orchestration and Marlin-based extrusion for continuous deposition on accessible hardware. A 50 Hz coordinator with acknowledgment-aware admission control holds the downstream planner non-empty (occupancy constrained to [8,12]) and collapses acknowledgment-latency tails from hundreds–thousands of milliseconds (naïve) to tens of milliseconds (windowed). Within a bench cell (robot bench-mounted; planar paths on a Voron plate; PETG, 0.48×0.20 mm), the policy yields visibly more continuous beads in qualitative coupons. These findings are intentionally scoped to planar, open-loop deposition and extruder-side timing; motion-coupled prints, conformal surfaces, and fieldbus transports remain future work. Under this scope, the evidence supports the claim that host-timed, ack-aware windowing is an effective and reproducible method for maintaining deposition continuity with ROS 2 + Marlin.

What was built and shown. A timer-driven coordinator at $\Delta t = 20ms$ (50 Hz) computes per-tick extrusion increments $\Delta E_k = \dot{E}(t_k)\,\Delta t \;\; with\; \dot{E} = \frac{w\,h\,v}{A_f}$ transmits small relative G1 EΔE commands, and admits new lines using an acknowledgment-aware in-flight window $[N_{min},\; N_{max}]$ = [8,12]. On an extruder-only bench (robot link connected but stationary to isolate transport effects), the method collapsed acknowledgment latency from 303 ms → 3.76 ms at 30 mm/s and 2255 ms → 3.73 ms at 60 mm/s, while bounding in-flight occupancy to $N \leq 12$ by design. The naïve tick-rate sender produced peaks > 700 lines and 241 starvation events at 60 mm/s; the windowed policy removed long-tail stalls and under a two-sided gate kept N inside the target band without starvation. These are precisely the timing conditions required for continuous bead formation on simple, planar paths.

How this answers the research objectives.

- **Integration.** The ROS 2 coordinator cleanly separates responsibilities, Marlin handles safety/thermals; the host enforces timing and admission for both transports (USB and TCP).
- **Continuity via timing control.** The data demonstrate that explicit, host-side pacing is sufficient to prevent planner starvation/overrun on commodity links—no firmware modification, fieldbus, or hard real-time OS required.

- **Reproducibility.** A set of parameters ($\boldsymbol{\Delta t}, [\boldsymbol{N_{min}}, \boldsymbol{N_{max}}], \boldsymbol{w}, \boldsymbol{h}, \boldsymbol{d_f}$) and a standard CSV schema (send/ack timestamps and in-flight counts) make results repeatable and comparable across setups.
- **Path to coupling.** The same node and period are used when streaming CRI motion; thus, the bench timing guarantees carry forward to robot-in-motion runs, with trajectory pacing layered on top.

Design lessons.

(i) Keep the host as the admission controller: parse "ok", hold a non-zero window, and never free-run at tick rate;

(ii) Choose N_{min} large enough to cover typical service time, e.g. $N_{min} \gtrsim \left\lceil \frac{p95(L_{ok})}{\Delta t} \right\rceil$;

(iii) Avoid host-side dwell during motion; if needed, apply mild bounded smoothing (3–5 ticks, ≤10 %) instead of stop-and-go;

(iv) Treat p_{99} L_{ok} exceeding ~50 ms as an operational warning;

(v) Keep thermal conditions stable (heatsink fan forced on when hot) so residual jitter does not translate into flow drift.

Limitations (clearly scoped). Evidence was captured on USB CDC with PETG at a single planar geometry window ($w = 0.48mm$ $h = 0.20mm$) and the timing study deliberately held robot motion static to isolate the host ↔ firmware stream. These constraints are acknowledged; they do not diminish the central claim that transport-level admission control is the lever that prevents starvation and backlog.

Impact and generality. The approach is hardware- and robot-agnostic so long as the downstream device acknowledges commands. Because it relies on geometric flow $Q(t) = w\,h\,v$ and a small in-flight window, it transfers naturally across materials, nozzles, and slicers; moving to deterministic transports (e.g., CAN/EtherCAT) is expected to tighten tails further without altering the control law. The thesis adapted its scope compared to the proposal: while a Gazebo-based digital twin was planned, real extruder–robot tests were executed instead. This choice delivered stronger practical validation of the synchronization method but left simulation-based scalability analysis to future work.

Future work. Couple the same controller to robot-in-motion coupons (straight line, square) and report bead-width statistics; evaluate arc moves (G2/G3) to reduce command rate;

compare firmware Linear Advance (M900) with host-side smoothing; and replicate the timing experiment over fieldbus to quantify transport effects. A lightweight feedback channel (pressure or vision) could then be layered on top for corners and transients. Conformal/tilted surfaces. Extend the host-timed controller to inclined paths and quantify timing → bead stability with changing tool normals. Motion-coupled coupons. Repeat naïve vs. windowed comparison during robot motion (straight line, square); add bead-width mean±SD. Transport variants. Re-run timing over CAN/EtherCAT to measure tail reduction vs. USB CDC. Future work should extend this bench validation into a full URDF/Gazebo environment with ros2_control integration, as originally outlined in the proposal, to assess scalability across multiple robot platforms and more complex toolpaths.

Closing statement. In sum, this thesis delivers a practical, open, and reproducible real-time synchronization method that closes the integration gap between ROS 2 motion execution and extrusion on low-cost hardware. By proving that a simple, acknowledgment-aware window at 50 Hz is enough to eliminate starvation and bound latency, it provides a solid foundation for reliable robotic deposition today and a clear, incremental path toward multi-axis, feedback-augmented printing tomorrow.

APPENDIX A - ROS 2 PACKAGE

Workspace Setup

```
~/ros2_ws/src/rmc/
  ├─ package.xml, setup.py, setup.cfg, CMakeLists.txt
  ├─ resource/rmc, rmc/__init__.py
  ├─ rmc/coordinator_node.py
  ├─ params.yaml
  ├─ marlin_extruder_preamble_petg.gcode
  ├─ square_plate.gcode
  └─ analyze_ack_csv.py
```

1. package.xml

```python
from setuptools import setup
package_name = 'rmc'
setup(
    name=package_name,
    version='0.1.0',
    packages=[package_name],
    data_files=[
        ('share/ament_index/resource_index/packages', ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        ('share/' + package_name, ['params.yaml']),
    ],
    install_requires=['setuptools','pyserial'],
    zip_safe=True,
    maintainer='you',
    maintainer_email='you@example.com',
    description='ROS 2 node coordinating IGUS ReBeL (CRI/TCP) and SKR/Marlin with ack-aware windowing.',
    license='MIT',
    entry_points={'console_scripts': ['coordinator = rmc.coordinator_node:main']},
)
```

2. setup.cfg

```
[develop]
script_dir=$base/lib/rmc
[install]
install_scripts=$base/lib/rmc
```

3. setup.py

```python
from setuptools import setup
package_name = 'rmc'
setup(
    name=package_name,
    version='0.1.0',
    packages=[package_name],
    data_files=[
        ('share/ament_index/resource_index/packages', ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        ('share/' + package_name, ['params.yaml']),
    ],
    install_requires=['setuptools','pyserial'],
    zip_safe=True,
    maintainer='rob',
    maintainer_email='rajbag4321@gmail..com',
    description='ROS 2 node coordinating IGUS ReBeL (CRI/TCP) and SKR/Marlin with ack-aware windowing.',
    license='MIT',
    entry_points={'console_scripts': ['coordinator = rmc.coordinator_node:main']},
)
```

4. params.yaml

```yaml
rebel_marlin_coordinator:
  ros__parameters:
    gcode_file: "/home/rob/Thesis/line_150mm.gcode"
    serial_port: "/dev/ttyACM0"
    baud: 115200
    robot_ip: "192.168.3.11"
    robot_port: 3920
    offset_x: 350.0
    offset_y: 0.0
    offset_z: 130.0
    speed: 30.0             # start safe; raise later
    dt: 0.02
    nmin: 8
    nmax: 12
    coast_sec: 0.04
    robot_min_gap: 0.06
    retry_limit: 5
    fan_pwm: 255            # single heatsink fan on FAN0 -> full
    line_w: 0.48
    layer_h: 0.20
    flow_mult: 1.0
    log_csv: "/home/rob/Thesis/ros2_ack_log.csv"
```

5. rmc/coordinator_node.py

```python
#!/usr/bin/env python3
# MIT License
# ReBeL-Marlin Coordinator (ROS 2, ack-aware window).
# - 50 Hz timer drives extrusion pacing (Δt)
# - window [Nmin,Nmax] on unacknowledged G-code lines keeps Marlin planner non-empty
# - streams robot Cartesian moves via CRI/TCP
# - publishes /coordinator/heartbeat and /coordinator/inflight for instrumentation
```

```python
import os, time, math, socket, threading, csv
import serial
import rclpy
from rclpy.node import Node
from std_msgs.msg import String, Int32

def now_mono(): return time.monotonic()

def parse_gcode_positions(path):
    pos = []; cur = {'X':0.0,'Y':0.0,'Z':0.0}; abs_mode=True
    with open(path,'r') as f:
        for raw in f:
            s = raw.strip()
            if not s or s.startswith(';'): continue
            if s.startswith('G90'): abs_mode=True;  continue
            if s.startswith('G91'): abs_mode=False; continue
            if s.startswith(('G0','G1')):
                nx=ny=nz=None
                for p in s.split()[1:]:
                    c=p[0].upper()
                    if c in 'XYZ':
                        try: v=float(p[1:])
                        except: continue
                        if c=='X': nx=v
                        if c=='Y': ny=v
                        if c=='Z': nz=v
                if abs_mode:
                    if nx is not None: cur['X']=nx
                    if ny is not None: cur['Y']=ny
                    if nz is not None: cur['Z']=nz
                else:
                    if nx is not None: cur['X']+=nx
                    if ny is not None: cur['Y']+=ny
                    if nz is not None: cur['Z']+=nz
                pos.append((cur['X'],cur['Y'],cur['Z']))
    return pos

def is_ok(s): return s.strip().lower().startswith("ok") and "resend" not in s.lower()

def drain(ser, buf):
    n = ser.in_waiting; lines=[]
    if n:
        buf += ser.read(n)
        parts = buf.split(b'\n'); buf = parts[-1]
        for p in parts[:-1]:
            try: t = p.decode(errors='ignore').strip()
            except: t = ""
            if t: lines.append(t)
    return buf, lines

def send_line(ser, line): ser.write((line.strip()+"\r\n").encode())
```

```python
class Coordinator(Node):
    def __init__(self):
        super().__init__('rebel_marlin_coordinator')

        p = self.declare_parameter
        self.gcode_file    = p('gcode_file', 'square_plate.gcode').value
        self.serial_port   = p('serial_port', '/dev/ttyACM0').value
        self.baud          = p('baud', 115200).value
        self.robot_ip      = p('robot_ip', '192.168.3.11').value
        self.robot_port    = p('robot_port', 3920).value
        self.offset_x      = float(p('offset_x', 350.0).value)
        self.offset_y      = float(p('offset_y',   0.0).value)
        self.offset_z      = float(p('offset_z', 220.0).value)
        self.speed         = float(p('speed', 30.0).value)
        self.dt            = float(p('dt', 0.02).value)
        self.nmin          = int(p('nmin', 8).value)
        self.nmax          = int(p('nmax',12).value)
        self.coast_sec     = float(p('coast_sec', 0.04).value)
        self.robot_min_gap = float(p('robot_min_gap', 0.06).value)
        self.retry_limit   = int(p('retry_limit', 5).value)
        self.fan_pwm       = int(p('fan_pwm', 255).value)
        self.log_csv       = p('log_csv', 'ros2_run_log.csv').value
        self.w             = float(p('line_w', 0.48).value)
        self.h             = float(p('layer_h',0.20).value)
        self.Af            = math.pi * (1.75/2)**2
        self.flow_mult     = float(p('flow_mult', 1.0).value)
        self.windowed      = bool(self.declare_parameter('windowed', True).value)


        if not os.path.exists(self.gcode_file):
            raise FileNotFoundError(f"G-code not found: {self.gcode_file}")
        self.positions = parse_gcode_positions(self.gcode_file)
        if len(self.positions) < 2:
            raise RuntimeError("No motion segments parsed.")
        self.seg_times = []
        geom_len = 0.0
        prev = self.positions[0]
        for pnt in self.positions[1:]:
            L = math.hypot(pnt[0]-prev[0], pnt[1]-prev[1])
            t = L / max(1e-6, self.speed)
            if self.coast_sec>0: t = max(0.0, t - self.coast_sec)
            self.seg_times.append(t); geom_len += L; prev = pnt

        bead_area = self.w * self.h
        E_rate = (bead_area * self.speed) / self.Af
        E_rate *= self.flow_mult
        self.dE = E_rate * self.dt
        self.F  = E_rate * 60.0
        self.ticks_total = max(1, int(round(sum(self.seg_times)/self.dt)))
        self.get_logger().info(
            f"Path={geom_len:.1f} mm | segs={len(self.seg_times)} | ΔE={self.dE:.5f} | F={self.F:.1f} | ticks={self.ticks_total}"
        )

        self.ser = serial.Serial(self.serial_port, self.baud, timeout=0, write_timeout=1)
        time.sleep(2.0); self.ser.reset_input_buffer()
        self.buf = b""
        self._send_preamble()


        self.csvf = open(self.log_csv, 'w', newline='')
        self.csvw = csv.writer(self.csvf)
        self.csvw.writerow(["t_send","sid_send","t_ok","sid_ok","inflight_after_send","inflight_after_ok"]); self.csvf.flush()

        self.buf = b""; self.inflight = 0; self.sent_ticks = 0; self.sid = 0
        self.sock = self._connect_robot()
        self._send_robot("CRISTART 2000 CMD MotionTypeCartBase CRIEND")

        self.extr_timer = self.create_timer(self.dt, self._extruder_cb)
        self.fan_timer  = self.create_timer(3.0, lambda: send_line(self.ser, f"M106 S{self.fan_pwm}"))

        self.robot_thread = threading.Thread(target=self._robot_loop, daemon=True)
        self.robot_thread.start()
        self.hb_pub = self.create_publisher(String, 'coordinator/heartbeat', 10)
        self.if_pub = self.create_publisher(Int32,  'coordinator/inflight',  10)
        self.hb_timer = self.create_timer(1.0, lambda: self.hb_pub.publish(String(data='alive')))
        def _hb_tick():
            self.hb_pub.publish(String(data='alive'))

        self.hb_timer = self.create_timer(1.0, _hb_tick)


    def _send_preamble(self):
        seq = ["M302 S220","M104 S235","M109 S235","M83","G92 E0","M155 S0"]
        for line in seq:
            send_line(self.ser, line)
            t0 = now_mono()
            while True:
                self.buf, ls = drain(self.ser, self.buf)
                if any(is_ok(x) for x in ls): break
                if now_mono() - t0 > 180: raise RuntimeError(f"Timeout waiting ok after: {line}")
                time.sleep(0.01)
        send_line(self.ser, f"M106 S{self.fan_pwm}")  # heatsink fan ON

    def _connect_robot(self):
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)
        s.settimeout(5.0)
        s.connect((self.robot_ip, self.robot_port))
        self.get_logger().info(f"Connected to ReBeL at {self.robot_ip}:{self.robot_port}")
        return s
```

```python
    def _recv_robot(self):
            if data and "ERROR" in data:
                self.get_logger().error(f"ROBOT ERROR: {data.strip()}"); return False
            return True
        except socket.timeout:
            return True

    def _send_robot(self, cmd):
        try:
            self.sock.sendall((cmd + "\r\n").encode("utf-8"))
            return self._recv_robot()
        except Exception as e:
            self.get_logger().warn(f"Robot socket issue: {e}")
            return False

    def _extruder_cb(self):
    # ---- Drain 'ok' acknowledgments ----
        self.buf, ls = drain(self.ser, self.buf)
        for s in ls:
            if is_ok(s) and self.inflight > 0:
                self.inflight -= 1
                # publish current occupancy for rqt_graph / debugging
                self.if_pub.publish(Int32(data=self.inflight))

        # ---- Finished? stop timer and cool down ----
        if self.sent_ticks >= self.ticks_total:
            self.destroy_timer(self.extr_timer)
            send_line(self.ser, "G1 E-0.8 F1800")
            send_line(self.ser, "M104 S0")
            return

        # ---- Admission policy ----
        if not self.windowed:
            # NAÏVE: always admit one line per timer tick
            send_line(self.ser, f"G1 E{self.dE:.5f} F{self.F:.1f}")
            self.inflight += 1
            self.sent_ticks += 1
            self.sid += 1
            self.if_pub.publish(Int32(data=self.inflight))
            return

        # WINDOWED: keep occupancy within [nmin, nmax]
        if self.inflight <= self.nmin:
            while self.inflight < self.nmax and self.sent_ticks < self.ticks_total:
                send_line(self.ser, f"G1 E{self.dE:.5f} F{self.F:.1f}")
                self.inflight += 1
                self.sent_ticks += 1
                self.sid += 1
                self.if_pub.publish(Int32(data=self.inflight))
        elif self.inflight < self.nmax:
            send_line(self.ser, f"G1 E{self.dE:.5f} F{self.F:.1f}")
            self.inflight += 1
            self.sent_ticks += 1
            self.sid += 1
            self.if_pub.publish(Int32(data=self.inflight))
    def _robot_loop(self):
        last_send = now_mono()
        tries = 0
        for (x, y, z), seg_t in zip(self.positions[1:], self.seg_times):
            xr, yr, zr = x + self.offset_x, y + self.offset_y, z + self.offset_z
            cmd = f"CRISTART 2000 CMD Move Cart {xr:.2f} {yr:.2f} {zr:.2f} 0 180 0 0 0 0 {self.speed:.1f} CRIEND"

            ok = self._send_robot(cmd)
            while not ok and tries <  self.retry_limit:
                tries += 1
                try: self.sock.close()
                except: pass
                time.sleep(0.3)
                self.sock = self._connect_robot()
                ok = self._send_robot(cmd)
            tries = 0

            gap = max(0.95*seg_t, self.robot_min_gap)
            sleep_for = (last_send + gap) - now_mono()
            if sleep_for > 0: time.sleep(sleep_for)
            last_send = now_mono()

        self.get_logger().info("Motion complete; cooling down.")
        for _ in range(60):
            send_line(self.ser, f"M106 S{self.fan_pwm}")
            time.sleep(1.0)
        send_line(self.ser, "M106 S0")
        self.csvf.flush(); self.csvf.close()
        self.get_logger().info("Job complete & cooled.")

def main():
    rclpy.init()
    node = Coordinator()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        try: node.sock.close()
        except: pass
        try: node.ser.close()
        except: pass
        node.destroy_node()
        rclpy.shutdown()
```

1. Environment

   OS: Ubuntu 22.04 LTS (64-bit).

   ROS 2: (Humble).

   Python: 3.10+.

   Dependencies: rclpy, pyserial, matplotlib (for analysis script).

   Hardware: IGUS ReBeL 6-DOF (CRI/TCP reachable), SKR Mini E3 V3.0 (USB CDC), E3D Revo Hemera XS, 24 V PSU.

APPENDIX B – G-CODE & ANALYSIS

1. marlin_extruder_preamble_petg.gcode

```
; PETG safe preheat + relative extrusion
M302 S220          ; PETG: forbid extrusion below 220 C
M104 S250          ; start heating
M109 S250          ; wait until 240 C
M83                ; relative extrusion
G92 E0             ; zero the extruder
M106 S80           ; part fan ~31% for PETG
; Now we start our host 50 Hz stream of: G1 E<ΔE> F<mm/min>
```

2. square_plate.gcode

```
; 20 cm perimeter, infill diagonals (single layer)
G21
G90
; perimeter
G1 X0 Y0 Z0.20 F6000
G1 X200 Y0 F6000
G1 X200 Y200 F6000
G1 X0   Y200 F6000
G1 X0   Y0
; diagonals (coarsely spaced)
G1 X0 Y0 F6000
G1 X200 Y200 F6000
G1 X200 Y0 F6000
G1 X0 Y200 F6000
; back to origin
G1 X0 Y0
```

3. Analyze_ack_csv.py

```python
import argparse, csv, io, math, os, statistics as st
import matplotlib.pyplot as plt

def read_text_sanitized(path: str) -> str:
    # Read raw bytes, strip NULs and decode forgivingly, normalize newlines
    with open(path, 'rb') as f:
        b = f.read()
    b = b.replace(b'\x00', b'')  # remove NULs
    s = b.decode('utf-8', errors='ignore')
    return s.replace('\r\n', '\n').replace('\r', '\n')

def sniff_delimiter(sample: str) -> str:
    # very simple sniffer
    first = sample.splitlines()[0] if sample else ""
    if first.count(';') > first.count(','): return ';'
    return ','

def norm_key(k: str) -> str:
    return k.strip().lower().replace(' ', '_')

def as_float(x):
    if x is None: return None
    x = str(x).strip()
    if x == '' or x.lower() == 'none': return None
    try: return float(x)
    except: return None

def as_int(x):
    f = as_float(x)
    try: return None if f is None else int(f)
    except: return None

def load_rows(path: str):
    text = read_text_sanitized(path)
    delim = sniff_delimiter(text)
    sio = io.StringIO(text)
    rdr = csv.DictReader(sio, delimiter=delim)
    rows = []
    # Normalize keys on each row
    for d in rdr:
        nd = {norm_key(k): v for k, v in d.items()}
        rows.append(nd)
    return rows, rdr.fieldnames

def pct(a, p):
    if not a: return float('nan')
    a = sorted(a)
    k = (len(a) - 1) * p
    f = math.floor(k)
    c = min(f + 1, len(a) - 1)
    return a[f] * (c - k) + a[c] * (k - f)
```

Auto-Import Completions: false Switch to
No inline suggestion available – Inline suggestio

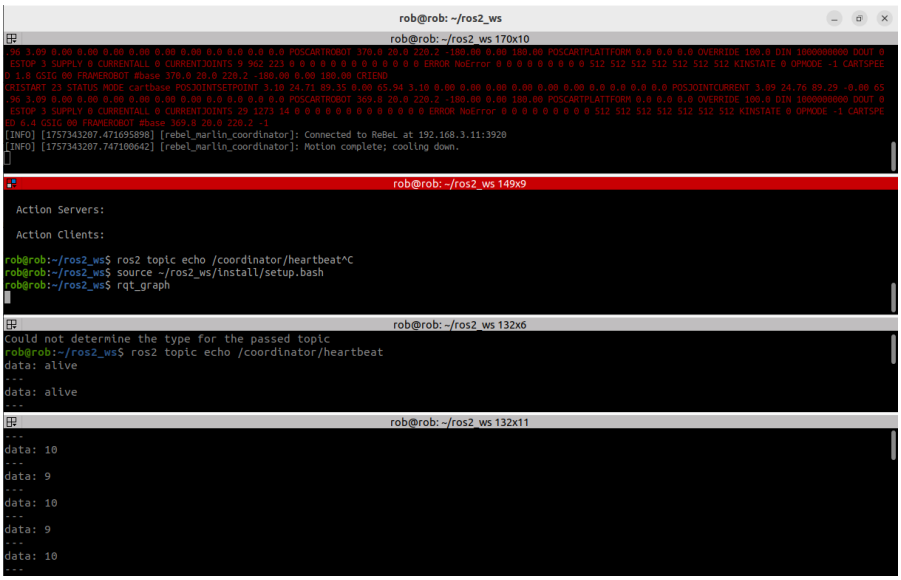
```python
def compute_stats(rows):
    # Accept both "in_flight_after_*" and "inflight_after_*"
    send_t = {}
    latencies = []
    occ_series = []

    for d in rows:
        ts  = as_float(d.get('t_send'))
        ss  = as_int  (d.get('sid_send'))
        to  = as_float(d.get('t_ok'))
        so  = as_int  (d.get('sid_ok'))
        ifs = as_int  (d.get('in_flight_after_send') or d.get('inflight_after_send'))
        ifo = as_int  (d.get('in_flight_after_ok')   or d.get('inflight_after_ok'))

        if ts is not None and ss is not None:
            send_t[ss] = ts
            if ifs is not None: occ_series.append(ifs)

        if to is not None and so is not None:
            if so in send_t:
                latencies.append(to - send_t[so])
            if ifo is not None: occ_series.append(ifo)

    stats = {}
    if latencies:
        stats['lat_med'] = st.median(latencies)
        stats['lat_p95'] = pct(latencies, 0.95)
        stats['lat_p99'] = pct(latencies, 0.99)
        stats['lat_max'] = max(latencies)
    if occ_series:
        stats['occ_p50'] = st.median(occ_series)
        stats['occ_p95'] = pct(occ_series, 0.95)
        stats['occ_max'] = max(occ_series)
        stats['starvations'] = sum(1 for v in occ_series if v == 0)
    stats['n_send'] = len([1 for d in rows if d.get('t_send') or d.get('sid_send')])
    stats['n_ok']   = len([1 for d in rows if d.get('t_ok')   or d.get('sid_ok')])
    return stats, latencies, occ_series

def plot_and_save(lat, occ, title_base, outdir=None):
    paths = []
    if lat:
        plt.figure()
        plt.hist(lat, bins=50)
        plt.title(f"Latency histogram — {title_base}")
        plt.xlabel("ack latency [s]"); plt.ylabel("count")
        if outdir:
            p = os.path.join(outdir, f"{title_base}_latency.png")
            plt.savefig(p, dpi=150, bbox_inches='tight'); paths.append(p)
    if occ:
        plt.figure()
        plt.plot(occ)
```

```python
        plt.plot(occ)
        plt.title(f"In-flight occupancy — {title_base}")
        plt.xlabel("event #"); plt.ylabel("in-flight")
        if outdir:
            p = os.path.join(outdir, f"{title_base}_inflight.png")
            plt.savefig(p, dpi=150, bbox_inches='tight'); paths.append(p)
    return paths

def main():
    ap = argparse.ArgumentParser(description="Analyze Marlin ack logs (robust).")
    ap.add_argument('csv', nargs='+', help='CSV files to analyze')
    ap.add_argument('--outdir', default='', help='Directory to save PNGs (optional)')
    ap.add_argument('--debug', action='store_true', help='Print detected fieldnames')
    args = ap.parse_args()

    outdir = args.outdir or ''
    if outdir and not os.path.isdir(outdir):
        os.makedirs(outdir, exist_ok=True)

    for path in args.csv:
        rows, flds = load_rows(path)
        base = os.path.splitext(os.path.basename(path))[0]
        print(f"\n== {path} ==")
        if args.debug:
            print("fields:", flds)
            print("rows:", len(rows))
        stats, lat, occ = compute_stats(rows)
        if not stats:
            print("(!) No usable rows found (headers may not match).")
            continue
        for k in ['n_send','n_ok','lat_med','lat_p95','lat_p99','lat_max','occ_p50','occ_p95','occ_max','starvations']:
            if k in stats:
                print(f"{k:>12}: {stats[k]:.4f}" if isinstance(stats[k], float) else f"{k:>12}: {stats[k]}")
        saved = plot_and_save(lat, occ, base, outdir if outdir else None)
        if saved:
            print("saved:", ", ".join(saved))
    if not args.outdir:
        plt.show()

if __name__ == '__main__':
    main()
```

4. Build & Run

```
rob@rob: ~/ros2_ws
rob@rob: ~/ros2_ws 80x24
rob@rob:~$ cd ~/ros2_ws
rob@rob:~/ros2_ws$ colcon build --symlink-install --packages-select rmc
Starting >>> rmc
Finished <<< rmc [2.19s]

Summary: 1 package finished [2.56s]
rob@rob:~/ros2_ws$ source ~/ros2_ws/install/setup.bash
rob@rob:~/ros2_ws$
```

```
rob@rob:~/ros2_ws$ source ~/ros2_ws/install/setup.bash
rob@rob:~/ros2_ws$ ros2 run rmc coordinator --ros-args   --params-file ~/ros2_ws
/src/rmc/params.yaml   -p windowed:=true
[INFO] [1757594240.302531168] [rebel_marlin_coordinator]: Path=150.0 mm | segs=1
 | ΔE=0.02395 | F=71.8 | ticks≈248
```



5. Availability

- Code & Data. The ROS 2 package (rmc), plotting scripts, parameter files, and the CSV logs used for all figures/tables are available at:

- GitHub: https://github.com/Raj-11-Bag/Bridging-the-gap-between-ROS-2-and-additive-manufacturing (release tag: v1.0-thesis) An archived snapshot is attached to the thesis submission.

ALPHABETICAL LIST OF REFERENCES

1. Open Robotics, Gazebo Simulation Guide: Building Complex Robot Worlds and Simulated Hardware, Open Robotics Documentation, 2023. Available: https://gazebosim.org/docs/latest/
2. Bonci, A., Francesco Gaudeni, Maria Cristina Giannini and Sauro Longhi (2023) 'Robot Operating System 2 (ROS 2)-based frameworks for industrial robotics: A survey', *Applied Sciences*, 13(23), 12796. https://doi.org/10.3390/app132312796
3. BIGTREETECH (2021) SKR Mini E3 V3.0 — Control board overview. https://bigtree-tech.com/blogs/news/new-release-bigtreetech-skr-mini-e3-v3-0-control-board
4. BIGTREETECH (GitHub) (2022) SKR Mini E3 V3.0 — Pinout (PDF). https://raw.githubusercontent.com/bigtreetech/BIGTREETECH-SKR-mini-E3/master/hardware/BTT%20SKR%20MINI%20E3%20V3.0/Hardware/BTT%20E3%20SKR%20MINI%20V3.0_PIN.pdf
5. Curry, A.T. (2021) *Additive manufacturing using robotic manipulators, FDM, and aerosol jet printers.* Master's thesis, University of Louisville. https://ir.library.louisville.edu/context/etd/article/5059/viewcontent/Masters_Thesis___Alex_Curry___final.pdf
6. Coogan, T.J. and Kazmer, D.O. (2019) 'Modeling of interlayer contact pressure in fused filament fabrication', *Journal of Rheology*, 63(4), 655–672. https://doi.org/10.1122/1.5093039
7. Coogan, T.J. and Kazmer, D.O. (2020) 'Prediction of interlayer strength in material extrusion additive manufacturing', *Additive Manufacturing*, 35, 101368. https://doi.org/10.1016/j.addma.2020.101368
8. Couto, M., Costa, R., Candea Leite, A., Lizarralde, F., Rodrigues, A. & da Cruz Payão Filho, J. (2020) 'Weld Bead Width Measurement in a GMAW WAAM System by using Passive Vision'. 10.48011/asba.v2i1.1121
9. CPR/CRI Manual V17 (PDF/manual citation)
10. Derekar, K.S. (2018) 'A review of wire arc additive manufacturing and advances in wire arc additive manufacturing of aluminium alloys', *Manufacturing Letters*, 15, 122–125. https://doi.org/10.1016/j.mfglet.2017.12.015

11. Ding, D., Pan, Z., Cuiuri, D., & Li, H. (2015) 'Wire-feed additive manufacturing of metal components: technologies, developments and future interests', *International Journal of Advanced Manufacturing Technology*. https://www.researchgate.net/publication/275973182_...
12. Ding (2023) "iWAAM—International Workshop on Additive Manufacturing (IFAC)". *IFAC-PapersOnLine*, 56(2), 6576–658. https://www.sciencedirect.com/science/article/pii/S2405896323006651
13. E3D-Online (2025) Revo Hemera XS — Support & specifications. https://e3d-online.com/pages/revo-support-hemera-xs
14. European Commission (n.d.) Electricity price statistics. https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Electricity_price_statistics
15. Fry, N.R., Richardson, R.C. and Boyle, J.H. (2020) 'Robotic additive manufacturing system for dynamic build orientations', *Rapid Prototyping Journal*, 26(4), 659–667. https://doi.org/10.1108/RPJ-09-2019-0243
16. F. Paus and N. Staub, "ROS 2 Control in Simulation and Real-World Applications: Bridging the Gap," Proceedings of the Simulation in Automation Symposium, 2022.
17. Gibson, I., Rosen, D., Stucker, B., & Khorasani, M. (2021) *Additive Manufacturing Technologies* (3rd ed.). Springer. https://doi.org/10.1007/978-3-030-56127-7
18. Huang, Q. et al. (2019) 'Generating robot trajectories for conformal 3D printing using non-planar layers', *Journal of Computing and Information Science in Engineering*, 19(4), 041003. https://doi.org/10.1115/1.4042796
19. Amazon product page: PETG filament. https://www.amazon.de/-/en/Filament-Dimensional-Accuracy-...
20. ISO (2012) *ISO 8373: Robots and robotic devices — Vocabulary.* https://standards.iteh.ai/...
21. ISO/ASTM (2015) *ISO/ASTM 52900: Additive manufacturing — Fundamentals and vocabulary.* https://www.iso.org/standard/74514.html
22. Ishak, I.B., Fisher, J. and Larochelle, P. (2016) 'Robot arm platform for additive manufacturing using multi-plane toolpaths', *ASME DETC/CIE 2016*. https://doi.org/10.1115/DETC2016-59438

23. igus / Commonplace Robotics (2024) CPR Robot Interface (CRI) — protocol & software overview. https://www.igus.eu/automation/robot-control-system/robot-software
24. Keating, S. & Oxman, N. (2013) 'Compound fabrication: A multi-functional robotic platform for digital construction', *Robotics and Computer-Integrated Manufacturing*, 29(6), 439–448. https://doi.org/10.1016/j.rcim.2013.05.001
25. Kramer, T., Proctor, F. & Messina, E. (2000) *The NIST RS274NGC Interpreter – Version 3*. NISTIR. https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=823374
26. Kuipers, T., van de Riet, K. and Doubrovski, E. (2020) 'Adaptive width control of contour-parallel toolpaths in FDM', arXiv:2004.13497. https://arxiv.org/abs/2004.13497
27. Kubalak, J.R. et al. (2016) 'Design and realization of a 6 degree of freedom robotic extrusion platform', *Solid Freeform Fabrication Symposium*, 1314–1332. https://repositories.lib.utexas.edu/...
28. Kubalak, J.R., Wicks, A.L. and Williams, C.B. (2018) 'Robotic deposition to enable multi-axis material extrusion', *TechConnect Briefs 2018*, 79–81. https://briefs.techconnect.org/...
29. Lu, K. (2025) 'Kinematic analysis and printing experiments of a multi-DOF robotic arm-based additive manufacturing', *Journal of Physics: Conference Series*, 3024(1), 012032. https://doi.org/10.1088/1742-6596/3024/1/012032
30. Luu, Q.K., La, H.M. and Ho, V.A. (2021) 'A 3-dimensional printing system using an industrial robotic arm', *IEEE/SICE SII 2021*, 443–448. https://doi.org/10.1109/IEEECONF49454.2021.9382645
31. Marlin Firmware (2025) G-code command index. https://marlinfw.org/meta/gcode/
32. Marlin Firmware (2025) Marlin 3D Printer Firmware (source). https://github.com/MarlinFirmware/Marlin
33. Marlin Firmware (2025) Marlin 2.1.x Documentation.
34. Maruyama, Y., Kato, S. and Azumi, T. (2016) 'Exploring the performance of ROS 2', *EMSOFT workshops*. https://ieeexplore.ieee.org/document/7743223
35. MIT License (2025). *The MIT License.* Open Source Initiative.

36. Nayyeri, P., Bougherara, H. and Zareinia, K. (2023) 'Development of a robotic additive manufacturing framework for fused deposition modeling', *CDSR'23*, Ottawa. https://doi.org/10.11159/cdsr23.216
37. OMG (2015) *Data Distribution Service (DDS) v1.4 Specification.* https://www.omg.org/spec/DDS/1.4/About-DDS/
38. Paul, S. (2024) 'Performance Evaluation of ROS2-DDS Middleware'. arXiv:2412.07485. https://doi.org/10.48550/arXiv.2412.07485
39. Pires, J.D.A.C. (2021) *Industrial robot-based 3D printer.* Master's thesis, Instituto Superior Técnico, Universidade de Lisboa. https://fenix.tecnico.ulisboa.pt/downloadFile/563345090420150/Thesis_83403.pdf
40. Rastegarpanah, M. (2024) 'Mobile robotics and 3D printing: addressing challenges in path planning and scalability', *Int. J. Computer Integrated Manufacturing.* https://doi.org/10.1080/17452759.2024.2433588
41. Rodríguez-Padilla, C., Ayala-Bravo, D. and Llamas, J. (2021) 'Algorithm for conformal 3D printing on non-planar surfaces', *Applied Sciences*, 11(16), 7509. https://doi.org/10.3390/app11167509
42. ROS Control Working Group, ros2_control Documentation – Design, Implementation, and API Reference, Open Robotics / ROS-control, 2024. Available: https://control.ros.org/
43. RTI (2021) *Understanding DDS Quality of Service (QoS).* https://community.rti.com/...
44. Schwicker, M. and Nikolov, N. (2022) 'Development of a fused deposition modeling system to manufacture non-planar structures', *Int. J. Mechanical Engineering and Robotics Research*, 11(2), 51–58. https://doi.org/10.18178/ijmerr.11.2.51-58
45. Stark, T. (2022) "Real-time execution management in the ROS 2 ecosystem." Doctoral dissertation, Saarland University. https://publikationen.sulb.uni-saarland.de/...
46. Voron Design (2025) "Voron V0.1 – SKR mini E3 V3.0 Wiring." https://docs.vorondesign.com/build/electrical/v0_miniE3_v30_wiring.html
47. V. Turlapov, ROS 2 Plugin Architecture: Debugging SystemInterface & ControllerManager in Complex Deployments, arXiv preprint arXiv:2304.11234, 2023. Available: https://link.springer.com/article/10.1007/s44267-023-00019-6

48. Wu, B., Zengxi Pan, Donghong Ding, Dominic Cuiuri (2018) 'A review of wire arc additive manufacturing: Processes, materials, and challenges', *Journal of Manufacturing Processes*, 35, 127–139. https://doi.org/10.1016/j.jmapro.2018.08.001
49. Wu, C., Dai, C., Fang, G., Liu, Y.-J. and Wang, C.C.L. (2017) 'RoboFDM: A robotic system for support-free fabrication using FDM', *ICRA 2017*, 1175–1180. https://doi.org/10.1109/ICRA.2017.7989140
50. Yao, Y., et al. (2021) '3D Printing of Objects with Continuous Spatial Paths by a Multi-Axis Robotic FFF Platform', *Applied Sciences*, 4–6.

LIST OF GENERATIVE AI TOOL USAGES

In this work, generative artificial intelligence tools were used for the following purpose:

Collection of conceptual explications on the topic of this work	☐
Generation of approaches towards scientific methodology or artistic development of this work	☑
Literature search	☑
Literature analysis	☑
Selection of methods and/or models	☐
Generation of code	☑
Generation of visualizations	☑
Generation of musical compositions and/or arrangements	☐
Structuring of the text of this work	☑
Formulation/Editing of the text of this work	☑
Translation of the text of this work	☐
Generation of presentation contents on the basis of this work	☑
Other:	☑

AFFIDAVIT

I hereby declare in lieu of an oath that I have independently written this work titled

Title: - Bridging the gap between ROS 2 and additive manufacturing: A Real-Time Control Approach for Continuous Deposition Tasks

and all related parts and that the work is in accordance with the version submitted in digital form. Only the sources and aids expressly named in the work have been used. I have marked as such any ideas taken over directly or in substance.

If generative AI tools were used to write the paper, this has been explicitly listed in the addendum at the end of this submission. Within the core text of my work, I have denoted all relevant contents that were created or paraphrased by generative AI (such as text passages, structures, images, and code or parts thereof) according to the citation style I used for short referencing secondary sources in my submission. In my list of references, each use of generative AI has been listed according to the citation style I used for full references, including the name of the tool, its version number, the date of retrieval, and the prompts used.

I hereby declare that I have not submitted this work in identical or similar form at this or another university previously.

srh
Affidavit, Version 07/10/2024

Berlin, 15/09/2025	Raj Manoj Bag
Location, Date	Name / Signature

LEGAL CAUTION

SRH Universities require an affidavit of independent authenticity of submitted scientific works in order to be assured that the signatory has generated the latter by him- or herself.

Since the law attributes a special relevance to affidavits and arising consequences thereof, submission of a false affidavit entails criminal punishment. Deliberately (and therefore knowingly) submitting a false affidavit may be punishable by up to three years imprisonment or a fine. Negligently submitting an affidavit (i.e. submitting despite having been able to recognize that the affidavit does not match factual truths) may be punishable by up to one year imprisonment or a fine.

The punitive frameworks are set forth in Section 156 of the German Penal Code (*falsche Versicherungen an Eides Statt*), as well as in Section 161 of the German Penal Code (*fahrlässiger Falscheid, fahrlässige falsche Versicherung an Eides Statt*).

Section 156 StGB: False declaration in lieu of oath

Whoever falsely makes a declaration in lieu of an oath before an authority which is competent to administer such declarations or falsely testifies whilst referring to such a declaration incurs a penalty of imprisonment for a term not exceeding three years or a fine.

Section 161 StGB: Negligent false oath; negligent false declaration in lieu of oath

(1) Whoever commits one of the offences referred to in sections 154 to 156 by negligence incurs a penalty of imprisonment for a term not exceeding one year or a fine.
(2) No penalty is incurred if the offender corrects the false statement in time. The provisions of section 158 (2) and (3) apply accordingly.

I hereby acknowledge the above

Berlin, 15/09/2025 | Raj Manoj Bag

Location, Date | Name Signature